

**KLASIFIKASI KEPADATAN LALU LINTAS
MENGUNAKAN *RANDOM FOREST* DAN SVM
BERBASIS *COMPUTER VISION* YOLO v8 DI JALAN
DIPONEGORO SURABAYA**

PROPOSAL SKRIPSI



DANI RULIANSYAH

NIM 434221059

PROGRAM STUDI D-IV TEKNIK INFORMATIKA

FAKULTAS VOKASI

UNIVERSITAS AIRLANGGA

TAHUN 2026

KATA PENGANTAR

Puji syukur ke hadirat Allah SWT atas segala rahmat dan karunia-Nya, sehingga penulis dapat menyelesaikan skripsi dengan judul “KLASIFIKASI KEPADATAN LALU LINTAS MENGGUNAKAN *RANDOM FOREST* DAN SVM BERBASIS *COMPUTER VISION* YOLOv8 DI JALAN DIPONEGORO SURABAYA”. Penulis juga ingin mengucapkan terima kasih yang sebesar-besarnya kepada:

1. Dekan Fakultas Vokasi Universitas Airlangga, Prof. Dr. Anwar Ma'ruf, M.Kes., drh., atas kesempatan yang diberikan untuk mengikuti pendidikan di Fakultas Vokasi Universitas Airlangga.
2. Eto Wuryanto Drs., DEA., selaku pembimbing utama, yang telah memberikan bimbingan, saran, dan dukungan sepanjang proses penulisan skripsi ini.
3. Anggota penguji yang telah memberikan masukan yang sangat berharga dalam penyelesaian skripsi ini.
4. Seluruh staf pengajar Program Studi Sarjana Terapan (S.Tr) Teknik Informatika Fakultas Vokasi Universitas Airlangga atas ilmu dan wawasan yang telah diberikan selama masa pendidikan.
6. Keluarga tercinta, yang telah memberikan doa, dan dukungan. Semoga skripsi ini dapat memberikan kontribusi positif di bidang teknologi kesehatan dan dapat bermanfaat bagi masyarakat luas.

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
DAFTAR TABEL.....	iii
DAFTAR GAMBAR.....	iv
BAB I PENDAHULUAN.....	6
1.1 Latar Belakang.....	6
1.2 Rumusan Masalah.....	8
1.3 Tujuan.....	9
1.4 Manfaat.....	9
1.5 Batasan Masalah.....	10
BAB 2 TINJAUAN PUSTAKA.....	12
2.1 Kepadatan Lalu Lintas.....	12
2.2 Computer Vision dan Object Detection.....	12
2.3 YOLO v8.....	13
2.4 Object Tracking (DeepSort).....	14
2.5 Preprocessing Data.....	15
2.6 Random Forest.....	18
2.7 Support Vector Machine.....	20
2.8 Hyperparameter Tuning.....	22
2.9 Evaluasi Kinerja Model.....	22
2.10 Active Learning dan Human-in-the-loop.....	25
2.11 Python dan Library.....	26
2.12 Flask.....	29
2.13 System Usability Scale (SUS).....	30
2.14 Penelitian Terdahulu.....	32
BAB III METODOLOGI PENELITIAN.....	34
3.1 Tempat dan Waktu Penelitian.....	34
3.2 Alat dan Materi.....	34
3.3 Metode Penelitian.....	37
DAFTAR PUSTAKA.....	49

DAFTAR TABEL

Tabel 2.1 parameter algoritma <i>Random Forest</i>	20
Tabel 2.2 parameter algoritma <i>Support Vector Machine</i>	22
Tabel 2.3 <i>confusion matrix</i>	24
Tabel 2.4 pertanyaan <i>System Usability Scale</i>	30
Tabel 2.5 penelitian terdahulu.....	32
Tabel 3.1 Spesifikasi <i>Hardware</i> pribadi.....	34
Tabel 3.2 Spesifikasi <i>Hardware</i> laptop teman.....	35
Tabel 3.3 Spesifikasi video rekaman cctv.....	35
Tabel 3.4 Spesifikasi <i>software</i> pribadi.....	36
Tabel 3.5 Spesifikasi <i>software</i> laptop teman.....	36
Tabel 3.6 Materi / Bahan penelitian.....	36
Tabel 3.7 contoh data setelah ekstraksi.....	39
Tabel 3.8 fitur lengkap dataset.....	39
Tabel 3.9 daftar fitur data beserta keterangan.....	39
Tabel 3.10 nilai parameter <i>Random Forest</i> yang akan diuji.....	42
Tabel 3.11 nilai parameter SVM.....	43
Tabel 3.12 timeline penelitian.....	55

DAFTAR GAMBAR

Gambar 3.1 Metode Penelitian.....	37
Gambar 3.2 Diagram <i>usecase</i>	45

BAB I

PENDAHULUAN

1.1 Latar Belakang

Kemacetan lalu lintas telah menjadi fenomena global yang kritis di kota-kota besar di seluruh dunia. INRIX 2025 Global Traffic Scorecard yang menganalisis 36 negara dan hampir 1.000 kota melaporkan bahwa 88% dari 290 kota di Amerika Serikat mengalami peningkatan kongesti, dengan pengemudi kehilangan rata-rata 49 jam per tahun dan kerugian ekonomi mencapai \$85,8 miliar. Sokido (2024) menyatakan bahwa kemacetan memberikan dampak langsung terhadap pertumbuhan ekonomi baik di negara maju maupun berkembang. Global Highways (2024) melaporkan bahwa Indonesia mengalami dampak signifikan dengan 125 juta kendaraan di jalan, meningkat 300% dari 30 juta kendaraan pada tahun 2004, dengan 85-90% berupa sepeda motor. Cambridge University Press (2024) menambahkan bahwa kota-kota besar mengalami penggunaan jalan melebihi kapasitas akibat urban sprawl ekstensif. Kompas (2025) melaporkan bahwa Surabaya menempati peringkat ke-70 dunia dan ke-4 Indonesia sebagai kota termacet dengan congestion level 31% dan waktu hilang 76 jam per tahun. Jawa Pos (2023), Kompas (2025), dan Suara Surabaya (2023) mengidentifikasi bahwa terdapat 25 titik kemacetan kritis di berbagai wilayah yang terjadi pada jam sibuk 06.00-09.00 dan 17.00-19.00.

Di antara berbagai titik kemacetan di Kota Surabaya, Jalan Diponegoro merupakan koridor strategis yang mengalami kepadatan tinggi sebagai jalur penghubung kawasan pusat kota dengan wilayah perkantoran, pendidikan, dan komersial. Timang et al. (2017) menjelaskan bahwa kawasan ini merupakan area dengan fungsi lahan beragam yang menghubungkan berbagai pusat aktivitas perkotaan. Pemilihan lokasi ini didasarkan pada karakteristik kemacetan yang didominasi oleh faktor volume kendaraan. Tanggara et al. (2021) melaporkan bahwa Jalan Diponegoro memiliki volume lalu lintas 5713,65 Skr/jam dengan derajat kejenuhan 0,7129, menunjukkan kepadatan yang murni berasal dari akumulasi volume kendaraan tanpa gangguan side friction signifikan, menjadikannya lokasi ideal untuk sistem klasifikasi berbasis volume kendaraan. Gao et al. (2025) menegaskan bahwa volume kendaraan merupakan indikator fundamental untuk mengukur kemacetan, didefinisikan sebagai jumlah kendaraan yang melewati titik tertentu dalam satuan waktu. Sebagai jalan arteri primer di kawasan perkotaan, Jalan Diponegoro dilalui oleh berbagai jenis kendaraan mulai dari sepeda motor, mobil pribadi, bus

angkutan umum seperti Suroboyo Bus, hingga Truk. Neamah & Karim (2024) menambahkan bahwa data volume memberikan informasi analitis lebih mendalam karena volume yang melebihi kapasitas jalan menyebabkan penurunan kecepatan dan kemacetan. Salam & Majid (2022) menyoroti bahwa komposisi jenis kendaraan juga krusial karena dalam kondisi heterogen seperti Indonesia, pengukuran harus memperhitungkan perbedaan dimensi dan kecepatan yang dikonversi ke smp.

Kemacetan lalu lintas menimbulkan dampak multidimensi yang signifikan terhadap berbagai aspek kehidupan perkotaan dan menjadi sumber permasalahan yang kompleks. Dari perspektif ekonomi, Kemenhub (2025) dan Republika (2025) melaporkan bahwa kerugian akibat kemacetan di lima kota besar Indonesia yaitu Semarang, Surabaya, Bandung, Medan, dan Makassar mencapai Rp 12 triliun per tahun, dimana data ini bersumber dari RPJMN (Rencana Pembangunan Jangka Menengah Nasional) 2020-2024 dan menunjukkan kerugian yang sudah melebihi APBD masing-masing kota tersebut. Dari segi lingkungan, kemacetan menyebabkan pencemaran udara dari emisi kendaraan yang bergerak dengan kecepatan rendah dan mengalami pembakaran bahan bakar tidak efisien. Boediningsih (2011) mengidentifikasi bahwa penurunan kualitas udara ini memicu berbagai penyakit seperti ISPA (infeksi saluran pernapasan akut), asma, bronkitis, dan gangguan pernapasan lainnya. Albalooshi (2025) menyatakan bahwa permasalahan utama yang muncul dari dampak-dampak tersebut adalah keterbatasan sistem manajemen lalu lintas yang masih mengandalkan data agregat dan pengamatan manual, sehingga tidak mampu memberikan respons yang cepat dan tepat terhadap kondisi dinamis lalu lintas. Oleh karena itu, diperlukan pendekatan sistematis berbasis data yang akurat dan real-time untuk mengidentifikasi, mengukur, dan mengklasifikasikan tingkat kepadatan lalu lintas secara objektif sebagai langkah awal dalam mengembangkan solusi mitigasi kemacetan yang efektif.

Untuk menerjemahkan data kuantitatif volume kendaraan menjadi informasi kualitatif tentang tingkat kepadatan lalu lintas, diperlukan pendekatan klasifikasi berbasis machine learning. Future Transportation (2025) menjelaskan bahwa klasifikasi adalah salah satu teknik supervised learning yang bertujuan memprediksi label kelas dari suatu instance berdasarkan fitur-fitur yang diekstrak, dimana dalam konteks lalu lintas, fitur berupa volume dan komposisi kendaraan dapat digunakan untuk mengklasifikasikan kondisi jalan ke dalam kategori seperti "Rendah", "Sedang", atau "Tinggi". Random Forest (RF) dan Support Vector Machine (SVM) merupakan dua algoritma klasifikasi yang telah terbukti efektif dalam prediksi kemacetan lalu

lintas. ElSahly & Abdelfatah (2023) dan IJRPR (2025) menyatakan bahwa Random Forest adalah algoritma ensemble learning yang menggabungkan prediksi dari banyak decision tree untuk meningkatkan akurasi, terbukti superior dalam menangani dataset kompleks dengan feature importance yang baik untuk interpretasi. Sementara itu, Ata et al. (2019) dan Scientific Reports (2024) menemukan bahwa SVM adalah algoritma yang mencari hyperplane optimal untuk memisahkan kelas-kelas data dengan margin maksimum, terbukti efektif dalam traffic classification dan traffic congestion control system. Kedua algoritma ini dipilih karena kemampuannya menangani data tabular dengan jumlah fitur terbatas namun memberikan hasil klasifikasi yang akurat dan interpretable.

Berdasarkan permasalahan dan potensi metode machine learning yang telah diuraikan, penelitian ini mengusulkan solusi berupa sistem klasifikasi kepadatan lalu lintas yang mengintegrasikan teknologi Computer Vision dengan algoritma Random Forest dan Support Vector Machine. Neamah & Karim (2024) menjelaskan bahwa sistem yang dikembangkan dapat dirancang untuk mengklasifikasikan tingkat kepadatan lalu lintas berdasarkan data volume dan komposisi kendaraan yang diekstrak secara otomatis dari rekaman video CCTV menggunakan algoritma YOLO v8 untuk deteksi objek dan DeepSORT untuk tracking kendaraan. Dalam penelitian ini, sistem dirancang untuk mengklasifikasikan tingkat kepadatan lalu lintas di Jalan Diponegoro Surabaya ke dalam tiga kategori (Rendah, Sedang, Tinggi) berdasarkan data hasil ekstraksi berupa jumlah kendaraan per kategori (sepeda motor, mobil, bus, truk) pada interval waktu tertentu yang kemudian digunakan sebagai input untuk melatih model klasifikasi Random Forest dan SVM. Tharwat (2020) menegaskan bahwa kedua model perlu dibandingkan performanya menggunakan matrix accuracy, precision, recall, dan F1-score untuk menentukan algoritma yang paling sesuai untuk klasifikasi kepadatan lalu lintas di lokasi studi. Sistem ini diharapkan dapat memberikan informasi kepadatan lalu lintas yang objektif, akurat, dan real-time untuk mendukung pengambilan keputusan manajemen lalu lintas di Kota Surabaya.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, terdapat kesenjangan (gap) antara kebutuhan akan data lalu lintas yang granular dan ketersediaan data di lapangan, serta perlunya sebuah sistem untuk menerjemahkan data mentah tersebut menjadi informasi yang bermakna. Untuk menjembatani kesenjangan tersebut, penelitian ini merumuskan beberapa pertanyaan utama yang akan dijawab:

1. Bagaimana implementasi sistem Computer Vision berbasis YOLOv8 untuk melakukan ekstraksi fitur volume dan komposisi kendaraan (motor, mobil, bus, dan truk) secara otomatis dari rekaman video pada studi kasus ruas jalan Diponegoro Surabaya?
2. Bagaimana hasil perbandingan akurasi dan performa antara algoritma Machine Learning Random Forest dan Support Vector Machine (SVM) dalam mengklasifikasi tingkat kepadatan lalu lintas (Rendah, Sedang, Tinggi) berdasarkan data volume kendaraan yang dihasilkan oleh sistem YOLO?
3. Bagaimana menyajikan hasil analisis kepadatan beserta *insight* komposisi lalu lintas Jalan Diponegoro untuk Dinas Perhubungan (Dishub) sebagai bahan pertimbangan pengambilan keputusan taktis?

1.3 Tujuan

Berdasarkan rumusan masalah yang telah dipaparkan, penelitian ini memiliki tujuan-tujuan spesifik sebagai berikut:

1. Mengimplementasikan sistem Computer Vision berbasis YOLO untuk mengekstraksi fitur volume dan komposisi kendaraan (motor, mobil, bus, dan truk) secara otomatis dari rekaman video pada studi kasus ruas Jalan Diponegoro Surabaya.
2. Menganalisis perbandingan akurasi dan performa algoritma Machine Learning Random Forest dan Support Vector Machine (SVM) dalam melakukan klasifikasi tingkat kepadatan lalu lintas (Rendah, Sedang, Tinggi).
3. Menghasilkan sebuah model klasifikasi tingkat kepadatan lalu lintas yang mampu menyajikan insight analitis mengenai pola kepadatan dan komposisi kendaraan sebagai bahan pertimbangan pengambilan keputusan untuk Dinas Perhubungan (Dishub).

1.4 Manfaat

Penelitian ini diharapkan dapat memberikan manfaat yang signifikan, baik secara praktis bagi instansi terkait maupun secara akademis bagi pengembangan ilmu pengetahuan.

1.4.1 Manfaat Praktis

Bagi Dinas Perhubungan (Dishub) Kota Surabaya:

1. Menyediakan sebuah prototipe sistem otomatis untuk pengumpulan data volume dan komposisi kendaraan secara granular (per 10 menit), menggantikan metode survei manual yang mahal dan memakan waktu.

2. Menghasilkan dataset analitis yang kaya (volume kendaraan, komposisi kendaraan, dan tingkat kepadatan) sebagai dasar ilmiah untuk pengambilan keputusan taktis, seperti penempatan petugas, evaluasi rekayasa lalu lintas, dan identifikasi pola jam sibuk yang spesifik.
3. Menjadi fondasi data yang dimana dapat menyajikan insight analitis mengenai pola kepadatan dan komposisi lalu lintas yang dapat dimanfaatkan sebagai bahan pertimbangan pengambilan keputusan taktis.

Bagi Masyarakat Umum:

Secara tidak langsung, hasil analisis dari penelitian ini dapat membantu perumusan kebijakan yang lebih baik oleh Dishub, yang bertujuan untuk mengurangi kemacetan dan pada akhirnya meningkatkan efisiensi serta kenyamanan mobilitas masyarakat.

1.4.2 Manfaat Akademis

Bagi Pengembangan Ilmu:

1. Memberikan studi kasus empiris mengenai penerapan metodologi terintegrasi yang menggabungkan Computer Vision (YOLO) untuk ekstraksi fitur dan Machine Learning (RF & SVM) untuk klasifikasi dalam domain transportasi urban.
2. Menyajikan analisis komparatif kinerja algoritma Random Forest dan SVM pada dataset lalu lintas dunia nyata (real-world) yang dikumpulkan secara primer, yang mungkin memiliki karakteristik noise dan variasi yang berbeda dari dataset benchmark standar.

Bagi Peneliti Selanjutnya:

Hasil penelitian dan dataset yang dihasilkan dapat dijadikan sebagai referensi dan baseline bagi penelitian selanjutnya yang berfokus pada analisis prediktif, pemodelan lalu lintas, atau pengembangan sistem ITS di Kota Surabaya

1.5 Batasan Masalah

Penelitian ini memiliki beberapa batasan yang ditetapkan untuk memperjelas ruang lingkup dan fokus penelitian. Batasan masalah dalam penelitian ini adalah sebagai berikut:

1. Lokasi dan Area Pengamatan

Penelitian ini hanya dilakukan pada Jalan Diponegoro Surabaya dengan pengamatan pada satu arah arus lalu lintas saja. Data yang digunakan merupakan rekaman video dari

1(satu) CCTV di Jalan Diponegoro selama 3 minggu (21 hari) dengan waktu pengamatan pada jam sibuk pagi (06.00-09.00 WIB) dan sore (15.00-20.00 WIB).

2. Data video yang digunakan dalam penelitian ini merupakan rekaman CCTV (recorded video) dengan interval waktu tertentu, sehingga sistem tidak melakukan pemrosesan arus lalu lintas secara langsung (live streaming real-time).

3. Algoritma deteksi objek

Algoritma yang digunakan dibatasi pada versi YOLO v8 dengan model pre-trained pada COCO dataset untuk klasifikasi jenis kendaraan (motor, mobil, bus, dan truk)

4. Kategori Kendaraan

Klasifikasi jenis kendaraan yang digunakan dalam penelitian ini terbatas pada empat kategori yaitu sepeda motor, mobil penumpang, bus, dan truk, sesuai dengan kemampuan deteksi model YOLO v8 pada COCO dataset.

BAB 2

TINJAUAN PUSTAKA

2.1 Kepadatan Lalu Lintas

Kepadatan lalu lintas (traffic density) didefinisikan sebagai jumlah kendaraan yang menempati suatu panjang jalan tertentu pada satu waktu, yang biasanya dinyatakan dalam satuan kendaraan per kilometer (kend/km). Rizvi & Ahmed (2025) dalam penelitian mereka tentang *Level of Service* (LOS) untuk urban arterials menyatakan bahwa kepadatan merupakan indikator spasial yang krusial untuk menentukan kualitas layanan jalan, yang mengkategorikan kondisi lalu lintas berdasarkan volume, kecepatan, dan waktu tempuh kendaraan. Dalam konteks manajemen transportasi modern, pemantauan kepadatan lalu lintas tidak lagi hanya bergantung pada pengukuran manual, melainkan beralih pada sistem transportasi cerdas (Intelligent Transportation Systems atau ITS) yang mampu menganalisis data lalu lintas secara real-time.

Rodriguez-Valencia et al. (2025) menjelaskan bahwa parameter utama yang diukur dalam ITS adalah volume arus lalu lintas (flow rate), yaitu jumlah kendaraan yang melewati titik tertentu dalam satuan waktu, serta klasifikasi jenis kendaraan yang melintas. Kondisi lalu lintas umumnya dikategorikan menjadi tiga tingkatan: Rendah (Low), Sedang (Medium), dan Tinggi (High), berdasarkan besaran volume kendaraan yang menempati ruang jalan. Menurut Khan et al. (2025) dalam penelitiannya tentang evaluasi traffic impact pada EDSA, klasifikasi tingkat kepadatan yang akurat sangat penting untuk sistem manajemen lalu lintas adaptif yang dapat memberikan respons real-time terhadap perubahan kondisi jalan. *Highway Capacity Manual* (HCM) 2022 menetapkan bahwa LOS ditentukan berdasarkan V/C ratio (*volume to capacity ratio*), dimana nilai V/C yang tinggi menunjukkan kepadatan lalu lintas yang semakin tinggi. Oleh karena itu, data volume kendaraan yang akurat dan terklasifikasi (mobil, motor, bus, truk) sangat diperlukan untuk menentukan label kepadatan tersebut secara presisi.

2.2 Computer Vision dan Object Detection

Computer Vision adalah cabang dari kecerdasan buatan (Artificial Intelligence) yang berfokus pada pengembangan sistem yang memungkinkan komputer untuk "melihat", memproses, dan menafsirkan gambar digital atau video selayaknya kemampuan visual manusia. Rahman et al. (2025) dalam survey komprehensif mereka menjelaskan bahwa dalam dekade

terakhir, penerapan Computer Vision telah menjadi komponen infrastruktur utama dalam berbagai aplikasi mulai dari autonomous driving hingga medical diagnostics, dengan kemajuan signifikan yang didorong oleh teknik deep learning yang mampu mengekstrak feature representations secara langsung dari data gambar.

Salah satu tugas utama dalam Computer Vision adalah Deteksi Objek (Object Detection). Chen et al. (2025) dalam penelitiannya tentang comprehensive review of object detection menyatakan bahwa berbeda dengan klasifikasi gambar sederhana yang hanya menentukan "apa" yang ada di dalam gambar, deteksi objek memiliki dua tujuan simultan: pengenalan (recognition) untuk mengidentifikasi kelas objek, dan lokalisasi (localization) untuk menentukan posisi spesifik objek yang direpresentasikan dengan bounding box. Metode modern dalam deteksi objek sangat bergantung pada teknologi Deep Learning, khususnya Convolutional Neural Networks (CNN), yang terbukti memiliki kemampuan ekstraksi fitur otomatis yang jauh lebih tangguh dibandingkan metode tradisional. Liu et al. (2024) menekankan bahwa perkembangan transformer-based models seperti Detection Transformer (DETR) dan Vision Transformers (ViTs) telah menghadirkan terobosan baru dalam object detection dengan kemampuan menangkap long-range dependencies dan contextual information yang lebih baik dibandingkan CNN konvensional, meskipun dengan kompleksitas komputasi yang lebih tinggi.

2.3 YOLO v8

Algoritma *You Only Look Once* (YOLO) merupakan terobosan dalam metode deteksi objek berbasis Deep Learning yang mengubah paradigma deteksi objek menjadi masalah regresi tunggal. Reis et al. (2023) dalam penelitiannya tentang real-time flying object detection menjelaskan bahwa berbeda dengan pendekatan two-stage detectors seperti R-CNN yang memisahkan proses pembuatan proposal wilayah dan klasifikasi, YOLO melakukan prediksi bounding box dan probabilitas kelas secara simultan dalam satu kali proses komputasi (single forward pass), memungkinkan YOLO mencapai kecepatan pemrosesan real-time dengan akurasi yang tetap kompetitif. YOLO membagi citra input menjadi grid berukuran $S \times S$, dimana setiap sel bertanggung jawab untuk mendeteksi objek jika pusat objek jatuh di dalam sel tersebut.

YOLOv8, yang dirilis oleh Ultralytics pada Januari 2023, memperkenalkan peningkatan signifikan dibanding pendahulunya. Teruel & Garcia (2023) dalam studi mendalam mereka tentang internal features YOLO v8 menyatakan bahwa YOLO v8 mengadopsi pendekatan anchor-free yang memprediksi pusat objek secara langsung tanpa memerlukan anchor boxes

yang ditentukan manual, sehingga meningkatkan fleksibilitas dan efisiensi deteksi terutama untuk objek dengan aspect ratio yang bervariasi. Sattar et al. (2025) dalam penelitiannya tentang enhancing YOLO v8 untuk online platform safety menunjukkan bahwa YOLO v8 menggunakan blok CSPNet (Cross Stage Partial Network) pada backbone-nya yang dikombinasikan dengan FPN+PAN *neck architecture* untuk ekstraksi fitur *multi-scale* yang lebih kaya namun tetap ringan secara komputasi. Ahmed & Khan (2025) dalam studinya tentang object detection in surveillance menekankan bahwa YOLO v8 dengan kemampuan real-time processing mencapai precision rates hingga 96.78% dan recall rates 96.89%, menjadikannya sangat efektif untuk mendeteksi kendaraan dengan variasi ukuran di jalan raya dalam kondisi yang kompleks.

$$Confidence = Pr(Object) \times IoU(pred, truth) \quad (2.1)$$

Keterangan :

Confidence : Tingkat kepercayaan deteksi objek (nilai antara 0-1)

Pr(Object) : Probabilitas bahwa cell mengandung objek (*objectness score*)

IoU(pred, truth) : *Intersection over Union* antara *predicted bounding box* dengan *ground truth bounding box*

2.4 Object Tracking (DeepSort)

Meskipun YOLO sangat efektif dalam mendeteksi objek pada setiap frame video, algoritma ini memiliki keterbatasan fundamental yaitu tidak memiliki memori antar frame. Oise et al. (2025) dalam penelitiannya tentang YOLO v8-*DeepSORT framework* menjelaskan bahwa ketika YOLO mendeteksi sebuah mobil pada frame ke-100 dan mobil yang sama muncul kembali pada frame ke-101, YOLO akan memperlakukannya sebagai dua deteksi terpisah tanpa mengetahui bahwa keduanya adalah objek yang sama, sehingga dalam konteks penghitungan kendaraan akan menyebabkan multiple counting dimana satu kendaraan dihitung berkali-kali saat bergerak melintasi area deteksi.

Untuk mengatasi masalah ini, diperlukan sistem Object Tracking yang mampu memberikan identitas unik (ID) kepada setiap objek yang terdeteksi dan mempertahankan identitas tersebut selama objek masih berada dalam frame video. *DeepSORT (Deep Simple Online and Realtime Tracking)* merupakan pengembangan dari algoritma SORT dengan penambahan deep learning features untuk meningkatkan kemampuan tracking dalam kondisi

oklusi dan kehilangan deteksi sementara. Menurut Li et al. (2022) dalam studinya tentang achieving adaptive visual multi-object tracking, *DeepSORT* menggunakan dua komponen utama: Kalman Filter untuk memprediksi posisi objek pada frame berikutnya berdasarkan pola pergerakan dengan mengatasi nonlinear errors melalui Unscented Kalman Filter (UKF), dan Deep Appearance Descriptor menggunakan CNN untuk mengekstrak fitur visual dari setiap objek yang terdeteksi. Labellerr (2025) dalam tutorial komprehensifnya menjelaskan bahwa *DeepSORT* menggunakan kombinasi Mahalanobis distance untuk motion consistency dan cosine distance untuk appearance similarity yang dihitung melalui Hungarian Algorithm untuk melakukan data association, sehingga mampu mengurangi ID switching hingga 45% dibandingkan *SORT* biasa. Kombinasi antara YOLO v8 untuk deteksi dan *DeepSORT* untuk tracking menciptakan sistem yang robust untuk mengekstrak data lalu lintas secara otomatis dan akurat dari video CCTV.

2.5 Preprocessing Data

Preprocessing merupakan tahap krusial dalam *machine learning* yang secara signifikan mempengaruhi kualitas dan performa model. Shadbahr et al. (2023) dalam penelitiannya yang dipublikasikan di *Communications Medicine* menekankan bahwa kualitas imputasi dan *preprocessing* data sangat penting ketika membangun model klasifikasi untuk data yang tidak lengkap, karena model yang dibangun dari data yang diproses dengan buruk dapat mengompromikan performa *classifier*. Dalam konteks penelitian ini, *preprocessing* data dilakukan untuk memastikan bahwa data hasil ekstraksi kendaraan dari video CCTV siap digunakan untuk *training model* Random Forest dan SVM.

2.5.1 Missing Values

Missing values merupakan masalah umum dalam *dataset real-world* yang dapat terjadi karena berbagai faktor seperti kegagalan sensor, kesalahan pengumpulan data, atau error dalam proses ekstraksi. Emmanuel et al. (2021) dalam survey komprehensif mereka yang dipublikasikan di *Journal of Big Data* menjelaskan bahwa missing values dapat terjadi karena faktor-faktor seperti *missing completely at random* (MCAR), *missing at random* (MAR), atau *missing not at random* (MNAR), dan mengabaikan atau menghapus missing values dapat menghasilkan analisis yang bias atau menyesatkan.

Metode penanganan *missing values* yang digunakan dalam penelitian ini adalah *median imputation* untuk *missing values* yang jumlahnya kurang dari 5% dari total data, dan penghapusan baris data untuk *missing values* yang lebih dari 5%. *Algorithms* (2023) dalam penelitiannya tentang *machine-learning-based imputation method* menyatakan bahwa *median imputation* menghasilkan data *imputed* yang lebih baik mempertahankan distribusi asli data, terutama pada data yang memiliki *outliers* atau *distribution* yang *skewed*.

$$\hat{x}_i = \text{median}(\{x_1, x_2, x_3, \dots, x_n\}) \quad (2.2)$$

Keterangan :

- $\hat{x}_i$: Nilai imputasi untuk mengisi missing value ke-i
 $\{x_1, x_2, x_3, \dots, x_n\}$: Semua nilai non-missing (valid) pada fitur yang sama
 $\text{median}()$: Fungsi untuk menghitung nilai tengah dari data yang telah diurutkan
 n : Jumlah data non-missing

2.5.2 Handling Outliers

Outliers adalah data point yang secara signifikan menyimpang dari pola umum dataset dan dapat berdampak besar pada hasil analisis. Dalam konteks penelitian ini, *outliers* dapat berupa jumlah kendaraan yang tidak wajar (terlalu tinggi atau terlalu rendah) dalam interval 10 menit yang mungkin disebabkan oleh error deteksi atau kondisi lalu lintas yang ekstrem.

Metode deteksi *outliers* yang digunakan adalah *Interquartile Range* (IQR), yang merupakan metode statistik *robust* untuk mendeteksi *outliers* terutama pada data yang tidak terdistribusi normal. Ahmed (2022) dalam penelitiannya yang dipublikasikan di *ARO-The Scientific Journal of Koya University* menjelaskan bahwa metode IQR secara aktif mengeliminasi efek dari rentang data yang tidak konsisten dan sangat efektif untuk menangani *outliers* pada data *skewed*.

$$\begin{aligned} \text{Lower Bound} &= Q1 - 1.5 \times IQR \\ \text{Upper Bound} &= Q3 + 1.5 \times IQR \\ IQR &= Q3 - Q1 \end{aligned} \quad (2.3)$$

Keterangan :

Lower Bound : Batas bawah untuk deteksi outlier

Upper Bound : Batas atas untuk deteksi outlier

Q1 : Kuartil pertama (persentil ke-25), nilai yang membagi 25% data terkecil

Q3 : Kuartil ketiga (persentil ke-75), nilai yang membagi 75% data terkecil

1.5 : Multiplier standar yang direkomendasikan

IQR : *Interquartile Range*, rentang antara *Q3* dan *Q1*

Sinsomboonthong (2022) dalam penelitiannya di *International Journal of Mathematics and Mathematical Sciences* menambahkan bahwa *IQR method* dengan multiplier 1.5 memberikan keseimbangan yang baik antara mendeteksi outliers yang genuine dan tidak menghilangkan data valid yang berada pada ekor distribusi.

2.5.3 Data Encoding

Data encoding adalah proses mengubah fitur kategorikal menjadi representasi numerik agar dapat diproses oleh algoritma *machine learning*. Potdar et al. (2020) dalam survey mereka yang dipublikasikan di *Journal of Big Data* menyatakan bahwa *neural networks* dan kebanyakan algoritma machine learning memerlukan input berupa vektor angka, sehingga diperlukan teknik untuk memetakan nilai kualitatif menjadi nilai numerik.

Label Encoding adalah teknik *encoding* yang paling *straightforward*, dimana setiap kategori unik menerima label integer. Hien et al. (2020) dalam penelitiannya di *International Journal of Advanced Computer Science and Applications* menjelaskan bahwa *label encoding* cocok untuk variabel kategorikal yang memiliki *ordinal relationship* (urutan yang bermakna). Alghazali et al. (2025) dalam penelitiannya di *International Journal of Computational Intelligence Systems* menambahkan bahwa *label encoding simple* dan *memory efficient*, serta cocok digunakan untuk *tree-based* models seperti Random Forest yang dapat menangani ordinal encoding tanpa menginterpretasikan jarak antar nilai sebagai meaningful distance.

2.5.4 Normalisasi

Normalisasi data adalah proses menskalakan fitur numerik ke rentang standar untuk memastikan bahwa semua fitur memiliki skala yang sebanding. Ahmed (2022) menekankan bahwa normalisasi data sangat penting terutama untuk algoritma yang sensitif terhadap skala

fitur, seperti SVM, *K-Nearest Neighbors*, dan *neural networks*, karena algoritma tersebut menggunakan perhitungan jarak dalam proses pembelajaran.

Min-Max Scaling adalah teknik normalisasi yang mentransformasi data ke rentang [0, 1] dengan mempertahankan distribusi relatif dari data asli. Sinsomboonthong (2022) dalam penelitiannya membandingkan performa berbagai teknik normalisasi untuk SVM dan menemukan bahwa min-max normalization memberikan *improvement* signifikan pada akurasi SVM dibandingkan tanpa normalisasi, dengan peningkatan akurasi hingga 10-15% pada beberapa dataset.

$$x_{norm} = \frac{(x - x_{min})}{(x_{max} - x_{min})} \quad (2.4)$$

Keterangan :

x_{norm} : Nilai data setelah normalisasi, hasil berada di rentang [0, 1]

x : Nilai data asli (nilai original dari fitur)

x_{min} : Nilai minimum dari fitur

x_{max} : Nilai maksimum dari fitur

Olawale (2025) menjelaskan bahwa tanpa normalisasi, fitur dengan rentang nilai yang besar (contoh: Total_Kendaraan 0-200) akan mendominasi perhitungan dot product dibandingkan fitur dengan rentang kecil (misalnya: Jam 0-23), membuat SVM bias terhadap fitur dengan skala besar dan menghasilkan decision boundary yang suboptimal. Normalisasi memastikan semua fitur memberikan kontribusi yang seimbang dalam proses pembelajaran SVM.

2.6 Random Forest

Random Forest adalah salah satu algoritma machine learning yang paling kuat dan fleksibel untuk berbagai masalah klasifikasi dan regresi. Schonlau & Zou (2020) dalam penelitiannya yang dipublikasikan di *Stata Journal* menyatakan bahwa algoritma ini merupakan metode ensemble yang menggabungkan hasil dari banyak decision tree untuk meningkatkan akurasi prediksi dan mengurangi risiko overfitting, dimana setiap pohon dilatih pada subset data yang berbeda melalui teknik bootstrap sampling. Salah satu keunggulan utama Random Forest adalah kemampuan internalnya untuk melakukan feature selection secara otomatis, dimana saat

membangun setiap pohon keputusan, algoritma secara acak memilih subset fitur sehingga hanya sebagian fitur yang digunakan untuk pemisahan di tiap node.

Proses kerja Random Forest dimulai dengan bootstrap sampling untuk membentuk beberapa subset data, kemudian untuk setiap subset dibangun sebuah decision tree dengan mempertimbangkan hanya sebagian acak dari fitur pada setiap node pembelahan. Rimal & Sharma (2025) dalam penelitian komprehensif mereka tentang ensemble machine learning menjelaskan bahwa setelah semua pohon dalam hutan terbentuk, Random Forest menggabungkan hasil prediksi: untuk masalah klasifikasi menggunakan voting mayoritas, sedangkan untuk regresi menggunakan rata-rata hasil prediksi dari setiap pohon, dengan hasil studi mereka menunjukkan Random Forest classifier mencapai akurasi 64% dalam multiclass classification yang outperform decision tree dan XGBoost. Khan & Ahmed (2024) dalam studinya tentang remote sensing based forest cover classification menekankan bahwa Random Forest sangat efektif dalam menangani dataset besar dan kompleks, serta mampu memberikan hasil yang stabil meskipun terdapat data yang hilang atau outlier, menjadikannya pilihan populer untuk berbagai aplikasi classification dan regression tasks.

$$Gini(t) = 1 - \sum (p_i)^2 \quad (2.5)$$

Keterangan :

$Gini(t)$: Nilai Gini *impurity* pada node t (nilai antara 0-1)

p_i : Proporsi sampel kelas i pada node t

Tabel 2.1 parameter algoritma Random Forest

No	Nama	Keterangan
1.	$n_estimators$	Jumlah pohon keputusan (decision trees) yang akan dibangun dalam forest. Semakin banyak pohon, model cenderung lebih stabil dan akurat.
2.	max_depth	Batas kedalaman maksimum setiap pohon.

No	Nama	Keterangan
		Digunakan untuk mengontrol kompleksitas model dan mencegah overfitting.
3.	<i>min_samples_split</i>	Jumlah minimum sampel yang diperlukan untuk membagi sebuah simpul (node) internal menjadi simpul baru.
4.	<i>min_samples_leaf</i>	Jumlah minimum sampel yang harus ada di setiap simpul daun (leaf node). Ini berfungsi untuk menghaluskan model.
5.	<i>random_state</i>	Menjamin hasil pembagian data dan pembangunan pohon yang konsisten setiap kali kode dijalankan (<i>reproducibility</i>).

2.7 Support Vector Machine

Support Vector Machine (SVM) merupakan salah satu metode supervised learning yang digunakan secara luas dalam tugas klasifikasi dan regresi. Martinez & Rodriguez (2024) dalam penelitiannya tentang distance-based kernel for SVM menjelaskan bahwa SVM bekerja dengan cara mencari hyperplane optimal yang memisahkan dua kelas data dengan margin maksimum, dimana prinsip dasar dari SVM adalah membangun pemisah (decision boundary) berdasarkan data pelatihan yang telah diberi label, dengan pemisah ini dirancang untuk memiliki generalisasi terbaik terhadap data yang belum pernah dilihat sebelumnya. Pemisahan ini dilakukan dengan mengoptimalkan jarak (margin) antara hyperplane dan data terdekat dari masing-masing kelas, yang disebut sebagai support vector.

SVM mampu menangani kasus klasifikasi baik yang linear maupun non-linear. Sharma et al. (2024) dalam studinya tentang ensemble and optimization algorithm in SVM menyatakan bahwa untuk data yang tidak dapat dipisahkan secara linear, SVM menggunakan fungsi kernel untuk mentransformasikan data ke dalam ruang berdimensi lebih tinggi, sehingga memungkinkan pemisahan linear di ruang tersebut. Wang & Li (2024) dalam comprehensive review mereka menjelaskan bahwa terdapat beberapa jenis kernel yang umum digunakan, antara

lain polynomial kernel, sigmoid kernel, dan Radial Basis Function (RBF) kernel. RBF merupakan kernel yang paling populer digunakan dalam kasus klasifikasi non-linear karena kemampuannya dalam mengukur kemiripan antar data berdasarkan jarak Euclidean, dengan hasil studi menunjukkan RBF kernel dengan optimization techniques menghasilkan peningkatan accuracy yang signifikan dibandingkan kernel lainnya untuk berbagai dataset klasifikasi.

$$w \cdot x + b = 0 \quad (2.6)$$

Keterangan :

w : *Weight vector*

x : *Feature vector*

b : *Bias term* (konstanta)

Tabel 2.2 parameter algoritma *Support Vector Machine*

No	Nama	Keterangan
1.	<i>kernel</i>	Fungsi matematika yang digunakan untuk mengubah data ke dimensi yang lebih tinggi agar dapat dipisahkan secara linear (contoh: Linear, RBF, Polynomial).
2.	C	Parameter penalti terhadap kesalahan klasifikasi. Nilai C yang besar akan mencoba mengklasifikasikan semua data dengan benar, namun berisiko overfitting.
3.	<i>gamma</i>	Menentukan seberapa jauh pengaruh satu sampel data terhadap pembentukan hyperplane. Nilai tinggi berarti hanya sampel dekat yang diperhitungkan.
4.	<i>random_state</i>	Menjamin hasil pembagian data dan pembangunan pohon yang konsisten setiap kali kode dijalankan (<i>reproducibility</i>).

2.8 Hyperparameter Tuning

Hyperparameter tuning adalah proses optimasi untuk menemukan kombinasi parameter terbaik yang menghasilkan performa model machine learning yang optimal. Berbeda dengan parameter model yang dipelajari secara otomatis dari data training, hyperparameter adalah konfigurasi yang harus ditentukan sebelum proses training dimulai dan sangat mempengaruhi kemampuan model dalam melakukan generalisasi. Ilemobayo et al. (2024) menjelaskan bahwa hyperparameter tuning merupakan langkah krusial dalam machine learning yang dapat secara dramatis meningkatkan akurasi prediksi model pada data yang belum pernah dilihat. Dalam konteks Random Forest, hyperparameter meliputi jumlah pohon ($n_estimators$), kedalaman maksimum pohon (max_depth), dan jumlah minimum sampel untuk split ($min_samples_split$), sedangkan untuk SVM meliputi jenis kernel ($kernel$), parameter regularisasi (C), dan parameter kernel ($gamma$).

GridSearchCV adalah salah satu metode *hyperparameter* tuning yang paling umum digunakan, dimana metode ini melakukan pencarian ekshaustif dengan mencoba semua kombinasi parameter yang ditentukan dan mengevaluasi performa setiap kombinasi menggunakan *cross-validation*. Franceschi et al. (2024) menyatakan bahwa *GridSearchCV* memberikan jaminan menemukan kombinasi parameter optimal dalam ruang pencarian yang telah didefinisikan, meskipun memerlukan waktu komputasi yang lebih lama karena harus mencoba semua kombinasi. Alternatif lain adalah *RandomizedSearchCV* yang melakukan sampling acak dari distribusi parameter, yang lebih efisien untuk ruang pencarian yang sangat besar. Egele et al. (2025) dalam penelitiannya mengenai *DeepHyper* menunjukkan bahwa metode *Bayesian optimization* dapat memberikan performa yang lebih baik dibandingkan grid search untuk kasus-kasus dengan ruang parameter yang kompleks, dengan melakukan sampling yang lebih cerdas berdasarkan hasil evaluasi sebelumnya.

2.9 Evaluasi Kinerja Model

Evaluasi kinerja model klasifikasi merupakan tahap penting untuk mengukur seberapa baik model *machine learning* dapat memprediksi kelas yang benar pada data yang belum pernah dilihat sebelumnya. Sebelum dilakukan evaluasi pada model *machine learning*, dilakukan evaluasi hasil ekstraksi YOLO v8 dari *Computer Vision* dengan menggunakan *Mean Absolute Percentage Error* (MAPE).

MAPE merupakan salah satu metrik evaluasi statistik yang digunakan untuk mengukur tingkat akurasi dari suatu model peramalan atau estimasi dalam bentuk persentase. MAPE mengukur seberapa besar penyimpangan antara data aktual dengan data hasil prediksi dalam nilai absolut, yang kemudian dibagi dengan nilai aktual untuk mendapatkan proporsi kesalahan (Chicco et al., 2021). Keunggulan utama MAPE dibandingkan metrik lain seperti Mean Squared Error (MSE) adalah sifatnya yang scale-independent, sehingga hasil evaluasi lebih mudah diinterpretasikan karena dinyatakan dalam satuan persen, tanpa dipengaruhi oleh skala besaran data yang diuji (Khair et al., 2021).

Dalam konteks penghitungan volume kendaraan, MAPE sangat relevan digunakan untuk memvalidasi sejauh mana hasil ekstraksi otomatis (seperti YOLO v8) mendekati hasil perhitungan manual oleh manusia sebagai ground truth. Secara matematis, rumus untuk menghitung nilai MAPE adalah sebagai berikut (Abbasniya et al., 2022):

$$MAPE = \frac{100\%}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right| \quad (2.7)$$

Keterangan :

y_i : Nilai aktual (*ground truth* / hasil pengamatan manual).

$\hat{y}_i$: Nilai prediksi (hasil ekstraksi sistem/YOLOv8)

n : Jumlah sampel data

$\sum$: Simbol penjumlahan dari data pertama hingga data ke- n

Akurasi model berdasarkan nilai MAPE dapat dikategorikan ke dalam empat kelompok kemampuan peramalan. Model dikatakan memiliki performa sangat baik jika nilai MAPE berada di bawah 10%, baik jika berada di rentang 10%–20%, cukup jika berada di rentang 20%–50%, dan buruk jika nilai MAPE melebihi 50% (Onditi et al., 2022).

Confusion Matrix adalah tabel kontingensi yang menampilkan perbandingan antara hasil prediksi model dengan nilai aktual dari data testing, dimana untuk klasifikasi multi-class berupa matriks $n \times n$ yang menunjukkan jumlah instance yang diprediksi sebagai kelas tertentu versus kelas aktualnya. Sathyanarayanan et al. (2024) menjelaskan bahwa confusion matrix menawarkan teknik yang mendalam dan terperinci untuk mengevaluasi performa classifier, yang

sangat penting dalam data science karena dapat mengungkap pola kesalahan prediksi yang tidak terlihat dari metrik akurasi sederhana.

Tabel 2.3 *confusion matrix*

<i>Actual Class</i>	<i>Predicted Class</i>	
	<i>Positive (1)</i>	<i>Negative (0)</i>
<i>Positive (1)</i>	TP	FN
<i>Negative (0)</i>	FP	TN

Keterangan :

TP : *True Positive*, Model memprediksi Positif, aktualnya Positif

TN : *True Negative*, Model memprediksi Negatif, aktualnya Negatif

FP : *False Positive*, Model memprediksi Positif, aktualnya Negatif (*Type I Error*)

FN : *False Negative*, Model memprediksi Negatif, aktualnya Positif (*Type II Error*)

Dari *confusion matrix*, dapat diturunkan empat kategori dasar untuk setiap kelas. *True Positive* (TP) untuk instance yang benar diprediksi sebagai kelas *positive*, *True Negative* (TN) untuk instance yang benar diprediksi sebagai kelas *negative*, *False Positive* (FP) untuk instance yang salah diprediksi sebagai kelas *positive*, dan *False Negative* (FN) untuk instance yang salah diprediksi sebagai kelas *negative*. Dari kategori tersebut dapat dihitung beberapa metrik evaluasi utama yang dijelaskan oleh Bhandari (2025):

- a. *Accuracy*, mengukur proporsi prediksi yang benar dari keseluruhan prediksi

$$Accuracy = \frac{(TP + TN)}{(TP + TN + FP + FN)} \quad (2.8)$$

- b. *Precision*, mengukur proporsi prediksi positif yang benar

$$Precision = \frac{TP}{(TP + FP)} \quad (2.9)$$

c. *Recall*, mengukur proporsi instance positif aktual yang berhasil terdeteksi

$$Recall = \frac{TP}{(TP + FN)} \quad (2.10)$$

d. *F1-Score*, adalah *harmonic mean* dari *precision* dan *recall*, yang sangat berguna ketika terdapat *imbalanced* data

$$F1 - Score = \frac{2 \times (Precision \times Recall)}{(Precision + Recall)} \quad (2.11)$$

Metrik-metrik ini memberikan perspektif berbeda tentang performa model, terutama pada kasus dataset yang tidak seimbang (*imbalanced dataset*) dimana jumlah sampel antar kelas sangat berbeda sehingga metrik akurasi saja tidak cukup informatif.

2.10 Active Learning dan Human-in-the-loop

Active Learning adalah paradigma dalam machine learning dimana model secara aktif memilih data mana yang paling informatif untuk diberi label, sehingga dapat mencapai performa tinggi dengan jumlah data berlabel yang lebih sedikit. Human-in-the-Loop (HITL) adalah pendekatan yang mengintegrasikan manusia dalam proses pembelajaran mesin, dimana manusia berperan sebagai oracle yang memberikan label atau koreksi pada prediksi model. Mosqueira-Rey et al. (2023) dalam survey komprehensif mereka menjelaskan bahwa terdapat berbagai jenis interaksi antara manusia dan algoritma machine learning, dimana Active Learning membuat sistem tetap memegang kontrol proses pembelajaran, Interactive Machine Learning memberikan interaksi yang lebih dekat antara pengguna dan sistem, sedangkan Machine Teaching memberikan kontrol penuh kepada pakar domain dalam proses pembelajaran.

Dalam konteks sistem machine learning yang terus berkembang, kombinasi Active Learning dengan HITL memungkinkan model untuk terus belajar dari data baru sambil memastikan kualitas data training melalui validasi manusia. Humans in the Loop (2025) menyatakan bahwa pendekatan HITL menggunakan active learning menawarkan transparansi yang lebih besar dengan melibatkan manusia dalam proses pembelajaran, memungkinkan pemahaman yang lebih baik terhadap pengetahuan, keraguan, dan kelemahan model. Wu et al. (2022) menekankan bahwa pendekatan HITL sangat penting untuk mencegah data poisoning atau error propagation, dimana prediksi yang salah dari model dapat masuk kembali ke dataset

training dan menurunkan performa model di masa depan. Oleh karena itu, validasi oleh pakar sebelum data hasil prediksi disimpan ke database utama merupakan langkah krusial untuk menjaga integritas ground truth dan memastikan model dapat beradaptasi dengan pola lalu lintas yang dinamis tanpa mengalami degradasi performa.

2.11 Python dan Library

Python telah menjadi bahasa pemrograman dominan dalam bidang data science, machine learning, dan computer vision karena sintaksnya yang sederhana, ecosystem library yang kaya, dan dukungan komunitas yang luas. Dalam penelitian ini, Python digunakan sebagai bahasa pemrograman utama untuk seluruh proses penelitian, mulai dari ekstraksi data, preprocessing, training model, hingga pengembangan aplikasi web. Berikut adalah penjelasan mengenai *library Python* yang digunakan dalam penelitian ini.

2.11.1 Python

Python adalah bahasa pemrograman tingkat tinggi yang pertama kali dikembangkan oleh Guido van Rossum pada tahun 1991. Carmatec (2025) menyatakan bahwa Python telah berevolusi dari framework routing dan templating dasar menjadi bahasa yang matang dengan hooks yang lebih dalam untuk perilaku global seperti customization points tingkat aplikasi untuk debugging dan contextual preservation. Python menggunakan sistem pengetikan dinamis (dynamic typing) dan sintaks yang dirancang menyerupai bahasa manusia, sehingga mempermudah programmer dalam memahami dan menulis kode. Keunggulan Python terletak pada keterbacaan kode yang tinggi, standard library yang komprehensif, dan ekosistem third-party packages yang sangat luas melalui Python Package Index (PyPI), menjadikannya pilihan utama untuk proyek data science dan artificial intelligence. Menurut ZestMinds (2025), Python tetap menjadi satu-satunya bahasa yang digunakan developer di seluruh dunia, dengan ekosistem yang terus berkembang untuk mendukung aplikasi AI dan sistem server modern.

2.11.2 Ultralytics

Ultralytics adalah *library Python* yang menyediakan implementasi *state-of-the-art* untuk algoritma YOLO (*You Only Look Once*), termasuk YOLOv8 yang digunakan dalam penelitian ini. Jocher et al. (2023) menjelaskan bahwa *Ultralytics* menyediakan API *high-level* yang memudahkan proses inference, training, dan fine-tuning model YOLO tanpa perlu memahami detail implementasi teknis yang kompleks. Library ini mendukung deteksi objek, segmentasi

instance, klasifikasi gambar, dan pose estimation dengan performa yang sangat baik. Ultralytics YOLOv8 hadir dengan model *pre-trained* pada COCO dataset yang mencakup 80 kelas objek termasuk kategori kendaraan (mobil, sepeda motor, bus, truk), sehingga dapat langsung digunakan untuk deteksi kendaraan tanpa perlu training ulang, atau dapat di-fine-tune untuk meningkatkan akurasi pada kondisi spesifik.

2.11.3 OpenCV

OpenCV (Open Source Computer Vision Library) adalah *library open-source* yang paling populer untuk *computer vision* dan *image processing*. Dalam penelitian ini, OpenCV digunakan untuk berbagai operasi pemrosesan video seperti membuka file video dari CCTV (cv2.VideoCapture), membaca *frame* demi *frame* (cap.read()), mengubah ukuran *frame* (cv2.resize), konversi warna (cv2.cvtColor), serta *drawing operations* seperti menggambar *bounding box* hasil deteksi (cv2.rectangle), menulis teks informasi *tracking* (cv2.putText), dan menggambar counting line (cv2.line). *OpenCV* versi terbaru yang digunakan dalam penelitian ini (*OpenCV 4.x*) menyediakan binding Python yang efisien dengan core library yang ditulis dalam C++, sehingga operasi komputasional dapat berjalan dengan cepat. Dokumentasi resmi *OpenCV* (2025) menunjukkan bahwa *library* ini terus berkembang dengan penambahan fitur-fitur *deep learning* dan optimasi untuk hardware modern termasuk GPU *acceleration*.

2.11.4 Pandas dan NumPy

NumPy (Numerical Python) adalah *library* fundamental untuk komputasi numerik di Python yang menyediakan struktur data *array* multidimensi (*ndarray*) dan berbagai fungsi untuk operasi matematika pada *array*. Harris et al. (2020) menjelaskan bahwa *NumPy* menjadi fondasi dari hampir semua *library scientific computing* di *Python* karena efisiensinya dalam menyimpan dan memanipulasi data numerik, dengan implementasi yang dioptimasi untuk operasi *array* yang kompleks. *Pandas* adalah *library* yang dibangun di atas *NumPy* untuk menyediakan struktur data dan fungsi yang memudahkan kerja dengan data terstruktur atau berbentuk tabel. *Pandas* menggabungkan kecepatan pemrosesan array dari *NumPy* dengan kemampuan manipulasi data yang fleksibel seperti pada *spreadsheet* dan *database* relasional. Dalam penelitian ini, *NumPy* digunakan untuk operasi array dan mathematical operations, sedangkan *Pandas* digunakan untuk menyimpan hasil penghitungan kendaraan dalam struktur *DataFrame*, melakukan rekapitulasi

data per interval waktu dengan fungsi *groupby* dan *resample*, serta ekspor hasil ke file CSV (*to_csv*).

2.11.5 Scikit-Learn

Scikit-Learn adalah *library machine learning* yang paling komprehensif untuk *Python*, menyediakan implementasi efisien dari berbagai algoritma klasik untuk klasifikasi, regresi, *clustering*, dan *dimensionality reduction*. Meskipun *paper original Scikit-Learn* dipublikasikan oleh Pedregosa et al. (2011), *library* ini terus berkembang dengan *update regular* hingga tahun 2025. Menurut dokumentasi resmi *Scikit-Learn* (2025), *library* ini dirancang dengan API yang konsisten dan mudah digunakan, serta terintegrasi dengan baik dengan *library scientific Python* lainnya seperti *NumPy* dan *Pandas*, dengan dukungan pendanaan dari berbagai institusi termasuk Microsoft, Quansight Labs, dan Chan-Zuckerberg Initiative yang memastikan pengembangan berkelanjutan. Dalam penelitian ini, *Scikit-Learn* digunakan untuk melatih *classifier Random Forest* (*RandomForestClassifier*) dan *Support Vector Machine* (SVC), membagi *dataset* menjadi *training* dan *testing* set dengan *stratified sampling* (*train_test_split*), melakukan *feature scaling* menggunakan *StandardScaler* atau *MinMaxScaler*, menghitung berbagai metrik evaluasi seperti *accuracy_score*, *precision_score*, *recall_score*, *f1_score*, dan *confusion_matrix*, serta melakukan *hyperparameter tuning* menggunakan *GridSearchCV*.

2.11.6 Joblib

Joblib adalah *library Python* yang menyediakan alat untuk *serialization* (penyimpanan) objek *Python*, terutama objek-objek berukuran besar seperti model *machine learning*. *Library* ini lebih efisien dibandingkan *pickle* bawaan *Python* untuk objek yang mengandung array *NumPy* berukuran besar, karena menggunakan kompresi dan optimasi khusus. Dalam penelitian ini, Joblib digunakan untuk menyimpan model *Random Forest* dan *SVM* yang telah dilatih ke dalam file dengan ekstensi *.pkl* (*joblib.dump*), sehingga model dapat di-load kembali (*joblib.load*) untuk melakukan prediksi tanpa perlu *training* ulang. Hal ini sangat penting untuk *deployment* aplikasi web, dimana model yang sudah dilatih dapat langsung digunakan untuk klasifikasi data baru dengan cepat dan efisien. Dokumentasi Joblib (2025) menunjukkan bahwa *library* ini terus diperbarui untuk mendukung *Python* versi terbaru dan integrasi dengan ekosistem *data science* modern.

2.11.7 Matplotlib

Matplotlib adalah library Python yang paling populer untuk membuat visualisasi data dua dimensi dalam bentuk plot, grafik, dan chart. *Library* ini dirancang untuk menghasilkan visualisasi yang *publication-quality* dengan kontrol yang sangat detail terhadap setiap elemen grafik, menyediakan *interface* yang mirip dengan MATLAB sehingga familiar bagi pengguna yang berasal dari *background engineering* dan *scientific computing*. Dalam penelitian ini, Matplotlib digunakan untuk membuat berbagai visualisasi seperti *confusion matrix* hasil evaluasi model, grafik *feature importance* dari Random Forest, plot learning curves untuk memonitor proses training, dan visualisasi distribusi data. Matplotlib versi terbaru (2025) telah mengalami peningkatan signifikan dalam performa rendering dan dukungan untuk visualisasi interaktif, serta dapat diintegrasikan dengan *Seaborn* untuk menghasilkan visualisasi statistik yang lebih menarik dan informatif.

2.11.8 SQLAlchemy

SQLAlchemy adalah *library Python* yang menyediakan *SQL toolkit* dan *Object-Relational Mapping (ORM)* untuk berinteraksi dengan database relasional. *SQLAlchemy* memungkinkan developer untuk bekerja dengan database menggunakan *high-level Python objects* tanpa perlu menulis *raw SQL queries*, namun tetap memberikan fleksibilitas untuk menggunakan *SQL* secara langsung jika diperlukan. Dalam penelitian ini, *SQLAlchemy* digunakan sebagai ORM untuk mengelola database *SQLite* yang menyimpan data hasil ekstraksi kendaraan dan hasil klasifikasi kepadatan lalu lintas. *SQLAlchemy* memfasilitasi operasi CRUD (Create, Read, Update, Delete) pada tabel database, mendukung fitur *migration* untuk perubahan skema database, dan menyediakan *connection pooling* untuk meningkatkan performa *query*. Integrasi *SQLAlchemy* dengan *Flask* melalui *extension Flask-SQLAlchemy* memudahkan pengelolaan database dalam aplikasi web. Dokumentasi *SQLAlchemy* (2025) menunjukkan bahwa versi terbaru mendukung *Python 3.12+* dengan peningkatan performa dan fitur *async/await* untuk operasi yang lebih efisien.

2.12 Flask

Flask adalah *micro web framework* untuk *Python* yang dikembangkan oleh Armin Ronacher, yang dirancang untuk membuat aplikasi web dengan cepat dan mudah dengan sedikit *boilerplate code*. Menurut dokumentasi resmi *Flask* (2025), *Flask* disebut "*micro*" *framework*

bukan karena fungsionalitasnya terbatas, melainkan karena core framework-nya dibuat seminimal mungkin namun dapat dengan mudah diperluas dengan *extensions* sesuai kebutuhan aplikasi, memberikan fleksibilitas tinggi kepada *developer* untuk mengorganisir kode sesuai arsitektur yang diinginkan. *Flask* bersifat *lightweight* dan modular dengan *dependencies* minimal, tidak memaksakan struktur aplikasi tertentu, serta menyediakan *built-in development server* yang memudahkan *testing* dan *debugging*.

Carmatec (2025) menjelaskan bahwa *Flask* versi 3.1.x yang diluncurkan akhir tahun 2024 menghadirkan *framework* yang lebih *robust* dan *customizable* dengan dukungan *async* yang lebih baik, mencerminkan evolusi *Python* itu sendiri. Dalam penelitian ini, *Flask* digunakan untuk membangun *web-based interface* yang memungkinkan pengguna berinteraksi dengan sistem klasifikasi kepadatan lalu lintas. Fungsi-fungsi utama yang diimplementasikan menggunakan *Flask* meliputi *upload interface* untuk mengunggah file CSV yang akan dianalisis, *real-time processing* status untuk menampilkan indikator proses saat data sedang diklasifikasi, *result visualization* untuk menampilkan hasil analisis dalam bentuk tabel statistik dan grafik visualisasi *trend* kepadatan lalu lintas, serta fitur prediksi kategori kepadatan (Rendah/Sedang/Tinggi) dari model *Random Forest* atau SVM. *Flask* menghubungkan *backend logic* (ekstraksi data, *training* model, dan klasifikasi) dengan *frontend interface*, memungkinkan *user-friendly interaction* dengan sistem yang kompleks.

2.13 System Usability Scale (SUS)

System Usability Scale (SUS) adalah kuesioner standar yang dikembangkan oleh John Brooke pada tahun 1986 untuk mengukur persepsi pengguna terhadap *usability* suatu sistem. UXtweak (2024) menjelaskan bahwa SUS terdiri dari 10 pertanyaan dengan skala *Likert* 1-5 yang dirancang untuk menilai berbagai aspek *usability* seperti kemudahan penggunaan, konsistensi *interface*, dan kepuasan pengguna secara keseluruhan. SUS telah menjadi salah satu instrumen evaluasi *usability* yang paling banyak digunakan karena kesederhanaannya, kemudahan administrasi, dan reliabilitas yang telah terbukti melalui berbagai studi validasi.

Tabel 2.4 pertanyaan System Usability Scale

No	Pertanyaan
1	Saya merasa akan sering menggunakan sistem ini

No	Pertanyaan
2	Saya merasa sistem ini terlalu rumit untuk digunakan
3	Saya merasa sistem ini mudah digunakan
4	Saya memerlukan bantuan dari orang teknis untuk menggunakan sistem ini
5	Saya merasa berbagai fungsi dalam sistem ini terintegrasi dengan baik
6	Saya merasa ada terlalu banyak inkonsistensi dalam sistem ini
7	Saya merasa kebanyakan orang akan belajar menggunakan sistem ini dengan cepat
8	Saya merasa sistem ini sangat rumit untuk digunakan
9	Saya merasa sangat percaya diri menggunakan sistem ini
10	Saya perlu belajar banyak hal sebelum dapat menggunakan sistem ini

Perhitungan skor SUS menggunakan formula standar yang menghasilkan nilai dengan rentang 0-100, dimana skor di atas 68 dianggap memiliki *usability* di atas rata-rata, dan skor di atas 80 dianggap *excellent*. Nielsen Norman Group (2025) menyatakan bahwa SUS adalah instrumen *post-test* yang diberikan kepada partisipan setelah seluruh sesi *usability testing* selesai atau setelah partisipan menyelesaikan semua tugas terkait sistem yang diuji. Interpretasi skor SUS dapat dipetakan ke dalam skala acceptability (*Not Acceptable, Marginal, Acceptable*) dan *grade scale* (A-F), yang memudahkan stakeholder dalam memahami tingkat *usability* sistem yang dikembangkan. Dalam konteks penelitian ini, evaluasi menggunakan SUS akan melibatkan pengguna sistem yaitu Admin dan *User* dari Dinas Perhubungan Surabaya, yang akan menilai kemudahan penggunaan aplikasi web untuk klasifikasi dan visualisasi data kepadatan lalu lintas.

$$\bar{x} = \frac{\sum x}{n} \quad (2.12)$$

Keterangan :

$\bar{x}$: Rata - rata

$\sum x$: Skor SUS

n : Jumlah user

2.14 Penelitian Terdahulu

Berikut merupakan penelitian terdahulu yang melakukan klasifikasi menggunakan Random Forest dan juga Support Vector Machine.

Tabel 2.5 penelitian terdahulu

No	Penulis (tahun) & Judul Penelitian	Metode	Hasil
1	Alfasi et al. (2025) "Neural Network-Based Prediction of Traffic Accidents and Congestion Levels Using Real-World Urban Road Data"	<i>Artificial Neural Network</i> (ANN) dengan Bayesian Optimization	
2	Rimal et al. (2025) "Comparative analysis of heart disease prediction using logistic regression, SVM, KNN, and random forest with cross-validation for improved accuracy"	Logistic Regression, SVM, KNN, Random Forest	RF: 91% SVM: 89% KNN: 87% LR: 85%
3	Saputra et al. (2025) "Comparison of Random Forest and SVM Algorithms in Diabetic Retinopathy Classification Based on Texture Features"	Random Forest & SVM	RF: 100% (10/10 test data) SVM: 95%

No	Penulis (tahun) & Judul Penelitian	Metode	Hasil
4	Withanage et al. (2024) “A performance evaluation of random forest, artificial neural network, and support vector machine learning algorithms to predict spatio-temporal land use-land cover dynamics”	Random Forest, SVM, ANN	RF: 96% (MOA) ANN: 96% SVM: 77-94%
5	IRJMETs (2025) “YOLOv8-Based Real-Time Vehicle Detection, Classification, and Counting System”	YOLO v8	Detection Accuracy: 96.8% Per-Class Accuracy: • Car: 97.5% • Bike: 95.3% • Bus: 98.1% • Truck: 96.2% Counting Performance: Counting Accuracy: 98.5% Processing Speed: • FPS: 30 frames/second • Detection Time: 32.5 ms/frame
6	Oise et al. (2025) “YOLOv8-DeepSORT: A High-Performance Framework for Real-Time Object Tracking and Counting”	YOLO v8 (detection) DeepSORT (tracking)	YOLO v8 mAP@0.5: 96.78% YOLOv8 Recall: 96.89% MOTA (Multiple Object Tracking Accuracy): 92.4%

BAB III

METODOLOGI PENELITIAN

3.1 Tempat dan Waktu Penelitian

Penelitian ini dilaksanakan di Lab Komputer Prodi D4 Teknik Informatika Universitas Airlangga pada periode Januari hingga Juni 2026. Data yang digunakan dalam penelitian ini berupa rekaman video CCTV Jalan Diponegoro Surabaya yang diperoleh dari Dinas Perhubungan Kota Surabaya, dengan periode perekaman pada bulan November hingga Desember 2025.

3.2 Alat dan Materi

Pencapaian tujuan penelitian ini didukung oleh penggunaan instrumen yang memadai untuk menjamin akurasi ekstraksi data serta performa model klasifikasi yang dihasilkan. Secara garis besar, kebutuhan penelitian ini terbagi menjadi dua aspek utama, yaitu ketersediaan perangkat keras yang mumpuni untuk pemrosesan citra digital secara intensif dan perangkat lunak berbasis deep learning serta machine learning untuk analisis data. Daftar alat dan materi yang digunakan dalam penelitian ini dijelaskan pada rincian berikut:

3.2.1 Alat

Penelitian ini menggunakan berbagai perangkat keras (hardware) dan perangkat lunak (software) sebagai berikut:

1. Hardware (Perangkat Keras)

Yang pertama ada perangkat pribadi yang nantinya akan digunakan mulai dari tahap *labeling data* hingga selesai:

Tabel 3.1 Spesifikasi *Hardware* pribadi

No	Nama	Spesifikasi
1.	<i>Processor</i>	Intel Core i5-8250U CPU @ 1.60 GHz (Turbo Boost 1.80 GHz)
2.	<i>Installed RAM</i>	8 GB DDR4 (4GB × 2, Dual Channel)
3.	<i>GPU</i>	NVIDIA GeForce MX130 2GB VRAM

No	Nama	Spesifikasi
4.	<i>Storage</i>	SSD 256 GB / HDD 1 TB

Kemudian terdapat spesifikasi untuk *hardware* laptop milik teman saya, yang saya pinjam untuk melakukan tahapan ekstraksi data :

Tabel 3.2 Spesifikasi *Hardware* laptop teman

No	Nama	Spesifikasi
1.	<i>Processor</i>	Intel Core i7-14700HX
2.	<i>Installed RAM</i>	16 GB DDR5
3.	<i>GPU</i>	NVIDIA GeForce RTX 4060 8GB DDR6
4.	<i>Storage</i>	1TB M.2 2242 PCIe Gen 4 SSD

Terdapat pula spesifikasi rekaman cctv yang saya dapat :

Tabel 3.3 Spesifikasi video rekaman cctv

No	Nama	Spesifikasi
1.	Resolusi rekaman	1280 x 720 pixel
2.	<i>Frame Rate</i>	30 fps

2. Software (Perangkat Lunak)

Perangkat lunak yang digunakan mencakup bahasa pemrograman Python versi 3.12.2 sebagai bahasa utama untuk seluruh proses penelitian. *Library* dan *framework* yang digunakan meliputi *Ultralytics* YOLOv8 untuk deteksi dan klasifikasi kendaraan dalam video, OpenCV (cv2) untuk pemrosesan video dan manipulasi frame, Pandas untuk manipulasi dan analisis data tabular, NumPy untuk operasi *array* dan komputasi numerik, *Scikit-learn* untuk implementasi algoritma Random Forest dan SVM serta *preprocessing* data dan evaluasi model, Flask untuk pengembangan aplikasi web,

Matplotlib/Seaborn untuk visualisasi data dan evaluasi model, serta *Joblib* untuk menyimpan dan memuat model yang telah dilatih. *Tools* pengembangan yang digunakan adalah *Visual Studio Code* sebagai *Integrated Development Environment* (IDE) utama dan *CapCut* untuk memotong dan memisahkan video per jam guna memudahkan proses ekstraksi data.

Tabel 3.4 Spesifikasi *software* pribadi

No	Nama	Spesifikasi
1.	Sistem operasi	Windows 10
2.	Jenis sistem	x64-based PC
3.	<i>Web server</i>	Laragon full 6.0
4.	Bahasa Pemrograman	Python 3.10

Tabel 3.5 Spesifikasi *software* laptop teman

No	Nama	Spesifikasi
1.	Sistem operasi	Windows 11
2.	Jenis sistem	x64-based PC
3.	<i>Web server</i>	Laragon full 6.0
4.	Bahasa Pemrograman	Python 3.12

3.2.2 Materi Penelitian

Materi penelitian atau objek data yang digunakan dalam penelitian ini terdiri dari dua komponen utama, yaitu:

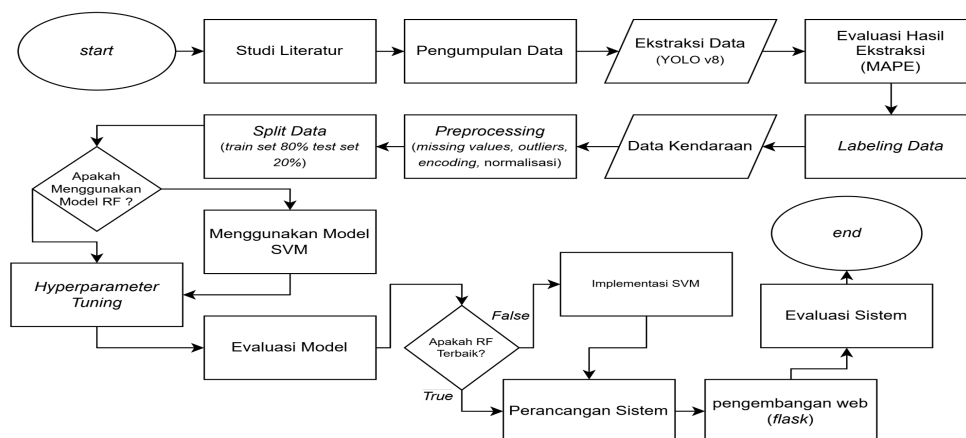
Tabel 3.6 Materi / Bahan penelitian

No	Nama	Keterangan
1	Data Video	Rekaman video CCTV lalu lintas yang diperoleh dari Dinas Perhubungan Kota Surabaya
2	Dataset Lalu Lintas	Dataset hasil ekstraksi video yang tersusun dalam format CSV sebanyak 1008 baris data.

3.3 Metode Penelitian

Metode penelitian ini menggambarkan alur kerja sistematis yang dilakukan untuk mencapai tujuan penelitian, yaitu mengembangkan sistem klasifikasi tingkat kepadatan lalu lintas berbasis computer vision dan machine learning. Penelitian ini menggunakan metode kuantitatif eksperimental dengan pendekatan supervised learning, dimana dua algoritma klasifikasi (Random Forest dan Support Vector Machine) dilatih menggunakan data historis volume dan komposisi kendaraan, kemudian dibandingkan performanya untuk menentukan model terbaik dalam mengklasifikasikan tingkat kepadatan lalu lintas.

Secara garis besar, penelitian ini terdiri dari 10 tahapan utama yang dimulai dari pengumpulan data video CCTV, ekstraksi data menggunakan algoritma YOLOv8 dan DeepSORT, pelabelan data oleh pakar, preprocessing data, pembagian data menjadi training dan testing set, training model dengan cross-validation, hyperparameter tuning untuk optimasi, evaluasi model menggunakan berbagai metrik, pemilihan model terbaik, dan terakhir deployment model ke dalam aplikasi web berbasis Flask. Berikut adalah diagram alur (flowchart) yang menggambarkan keseluruhan rancangan penelitian:

**Gambar 3.1 Metode Penelitian**

Setiap tahapan dalam flowchart di atas akan dijelaskan secara rinci pada subbab-subbab berikut:

3.3.1 Studi Literatur

Studi literatur dalam penelitian ini bertujuan untuk mendalami metode computer vision menggunakan algoritma YOLOv8 untuk deteksi dan klasifikasi kendaraan, serta algoritma machine learning Random Forest dan Support Vector Machine untuk klasifikasi tingkat kepadatan lalu lintas. Kajian literatur mencakup penelusuran teori dasar tentang kepadatan lalu lintas dan indikator teknisnya, penelitian terdahulu yang menggunakan pendekatan serupa, serta best practices dalam implementasi sistem klasifikasi berbasis data visual dan numerik. Referensi utama yang dikaji meliputi jurnal ilmiah, buku teks, dan dokumentasi teknis yang relevan dengan topik penelitian, yang kemudian menjadi landasan teoretis dalam perancangan sistem klasifikasi kepadatan lalu lintas.

3.3.2 Pengumpulan Data (Data Collection)

Tahap pertama adalah pengumpulan data berupa rekaman video CCTV Jalan Diponegoro Surabaya yang diperoleh dari Dinas Perhubungan Kota Surabaya. Pemilihan satu titik lokasi penelitian di Jalan Diponegoro didasarkan pada pertimbangan kompleksitas prosedur pengolahan data dan optimalisasi sumber daya. Proses ekstraksi data volume kendaraan menggunakan algoritma YOLOv8 melibatkan pengolahan data video beresolusi tinggi yang membutuhkan durasi komputasi yang intensif. Selain itu, tahap validasi akurasi ekstraksi melalui penghitungan manual (ground truth). Oleh karena itu, penelitian ini difokuskan pada satu lokasi strategis guna menjamin integritas dan reliabilitas data yang dihasilkan, sehingga model yang dikembangkan memiliki tingkat akurasi yang optimal sebelum diimplementasikan pada skala yang lebih luas.

Video diambil dari salah satu cctv pada Jalan Diponegoro untuk periode 3 minggu (21 hari) dengan durasi 8 jam per hari, mencakup jam sibuk pagi (06.00-09.00 WIB) dan sore (15.00-20.00 WIB). Setiap hari dalam seminggu (Senin-Minggu) diulang sebanyak 3 kali untuk memastikan variabilitas dan konsistensi data. Video yang terkumpul kemudian dipisahkan per jam menggunakan software CapCut untuk memudahkan proses ekstraksi data pada tahap selanjutnya.

3.3.3 Ekstraksi Data (Data Extraction)

Pada tahap ini, video yang telah dipisahkan per jam diproses menggunakan algoritma YOLOv8 untuk mendeteksi dan mengklasifikasikan kendaraan menjadi 4 kategori: sepeda

motor, mobil, bus, dan truk. Setiap kendaraan yang terdeteksi kemudian di-tracking menggunakan algoritma DeepSORT untuk memberikan ID unik dan memastikan setiap kendaraan hanya dihitung satu kali saat melewati counting line (garis virtual).

Proses ekstraksi menggunakan library OpenCV untuk membaca frame video dan Ultralytics YOLOv8 untuk inference model. Hasil deteksi dan tracking disimpan dalam file log CSV dengan struktur data sebagai berikut:

Tabel 3.7 contoh data setelah ekstraksi

Hari	Jam	Menit	Motor	Mobil	Bus	Truk	total_kendaraan
Senin	7	10	30	19	3	11	63

Data diekstrak dengan interval 10 menit, sehingga dari 8 jam pengamatan per hari menghasilkan 48 baris data, dan total 3 minggu menghasilkan 1.008 baris data.

3.3.4 Evaluasi Hasil Ekstraksi

Sebelum hasil ekstraksi data volume kendaraan digunakan sebagai input untuk model klasifikasi kepadatan, dilakukan proses validasi untuk mengukur tingkat akurasi algoritma YOLOv8 dalam menghitung dan mengklasifikasikan kendaraan. Mengingat volume data yang besar, validasi dilakukan menggunakan metode *Comparison with Ground Truth* pada sampel data uji (random sampling).

Prosedur evaluasi dilakukan dengan memilih secara acak sampel video berdurasi 10 menit yang mewakili kondisi pencahayaan dan tingkat kepadatan yang berbeda (pagi, siang, dan sore). Data jumlah kendaraan hasil deteksi otomatis (*System Count*) dibandingkan dengan hasil perhitungan manual (*Manual Count*) yang dilakukan oleh pengamat manusia sebagai ground truth. Kinerja algoritma ekstraksi kemudian diukur menggunakan metrik *Mean Absolute Percentage Error* (MAPE) untuk mengetahui persentase rata-rata penyimpangan hasil hitung sistem terhadap kondisi aktual dengan menggunakan persamaan (2.19), serta *Confusion Matrix* untuk mengevaluasi ketepatan klasifikasi jenis kendaraan. Nilai MAPE yang rendah mengindikasikan bahwa data hasil ekstraksi memiliki reliabilitas yang tinggi untuk digunakan pada tahap pemodelan selanjutnya.

3.3.5 Pelabelan Data (Data Labeling)

Setelah data hasil ekstraksi diperoleh, dilakukan Proses pelabelan data (ground truth) dilakukan melalui metode observasi visual manual oleh tenaga ahli dari Dinas Perhubungan Kota Surabaya terhadap rekaman CCTV berdurasi 10 menit. Penentuan label kelas kepadatan tidak didasarkan pada ambang batas jumlah kendaraan (threshold) yang kaku, melainkan mengacu pada karakteristik arus lalu lintas secara kualitatif. Tingkat kepadatan dikategorikan menjadi 3 kelas dengan rincian pada tabel seperti berikut :

Tabel 3.8 standar pelabelan data

No	Nama Label	Keterangan
1	Rendah	Arus : Bebas (free flow), kecepatan maksimal, manuver tinggi. Volume : Jauh di bawah kapasitas jalan; tidak ada antrian.
2	Sedang	Arus: Stabil tapi mulai terbatas jarak antar kendaraan merapat. Volume: Meningkat namun masih mengalir lancar tanpa hambatan berarti.
3	Tinggi	Arus: Tidak stabil, kecepatan sangat rendah, sering stop-and-go. Volume: Mendekati/melebihi kapasitas; penumpukan antrian yang signifikan.

Hasil pelabelan ditambahkan sebagai kolom baru "Tingkat_Kepadatan" dalam file CSV, sehingga struktur data final menjadi :

Tabel 3.9 fitur lengkap dataset

Hari	Jam	Menit	Motor	Mobil	Bus	Truk	total_kendaraan	tingkat_kepadatan
Senin	7	10	30	19	3	11	63	Sedang

Tabel 3.10 daftar fitur data beserta keterangan

No	Nama	Keterangan
1	hari	Menunjukkan nama hari pengambilan data (Senin-Minggu) untuk menangkap pola perbedaan lalu lintas harian

No	Nama	Keterangan
2	jam	Rentang waktu dalam satuan jam (00-23) yang merepresentasikan periode pengambilan data.
3	menit	Penanda interval waktu dalam menit (agregasi per 10 menit) untuk analisis data yang lebih granular.
4	motor	Jumlah kendaraan roda dua yang terdeteksi oleh YOLO
5	mobil	Jumlah kendaraan roda empat (kendaraan pribadi) yang terdeteksi YOLO
6	bus	Jumlah kendaraan angkutan umum atau kendaraan besar kategori bus yang terdeteksi.
7	truk	Jumlah kendaraan angkutan barang atau kendaraan berat yang terdeteksi
8	total	Akumulasi keseluruhan jenis kendaraan (Motor + Mobil + Bus + Truk) dalam satu unit waktu.
9	tingkat_kepadatan	Label kategori (<i>target variable</i>) yang menunjukkan kondisi jalan (Rendah, Sedang, Tinggi)

3.3.5 Preprocessing Data

Sebelum data digunakan untuk training model, dilakukan beberapa tahapan preprocessing untuk memastikan kualitas dan kesesuaian data. Preprocessing data merupakan tahap krusial yang secara signifikan mempengaruhi performa model machine learning, sebagaimana dijelaskan pada subbab 2.5. Tahapan preprocessing yang dilakukan dalam penelitian ini meliputi handling missing values, handling outliers, data encoding, dan normalisasi data.

1. Handling Missing Values

Memeriksa dan menangani data yang hilang atau kosong (missing values) yang mungkin terjadi akibat kegagalan deteksi atau error dalam proses ekstraksi. Sebagaimana dijelaskan pada subbab 2.5.1, missing values dapat menimbulkan bias dalam analisis data jika tidak ditangani dengan tepat.

- Missing values < 5%: Dilakukan imputation menggunakan median dari nilai non-missing pada fitur yang sama, menggunakan Persamaan (2.2).

- Missing values $\geq 5\%$: Baris data yang mengandung missing values dihapus (*drop*) untuk menghindari bias yang berlebihan.

Alasan penggunaan median, karena lebih *robust* terhadap outliers dibandingkan mean, sehingga mampu mempertahankan distribusi asli data dengan lebih baik, terutama pada data yang memiliki *distribution* yang *skewed*.

2. Handling Outliers

Mendeteksi dan menangani *outliers* (data pencilan) yang tidak wajar, seperti jumlah kendaraan yang terlalu tinggi atau terlalu rendah dalam 10 menit. Metode yang digunakan adalah IQR (*Interquartile Range*), dengan menggunakan persamaan (2.3).

- *Outlier* di-replace dengan nilai batas (*capping*) atau dihapus jika jumlahnya sedikit

3. Data Encoding

Mengubah fitur kategorik menjadi numerik agar dapat diproses oleh algoritma *machine learning Random Forest* dan SVM. Sebagaimana dijelaskan pada subbab 2.5.3, algoritma *machine learning* memerlukan input berupa vektor angka.

- Hari: Encoding menjadi angka (Senin=1, Selasa=2, ..., Minggu=7) menggunakan Label Encoding
- Tingkat_Kepadatan: Encoding menjadi angka (Rendah=0, Sedang=1, Tinggi=2) menggunakan Label Encoding

Penggunaan *Label encoding* cocok untuk *tree-based models* seperti *Random Forest* yang dapat menangani *ordinal encoding* tanpa menginterpretasikan jarak antar nilai sebagai *meaningful distance*. Encoding ini juga simple dan memory-efficient.

4. Normalisasi Data

Melakukan normalisasi fitur numerik menggunakan Min-Max Scaling untuk menskalakan nilai ke rentang $[0, 1]$. Sebagaimana dijelaskan pada subbab 2.5.4, normalisasi ini sangat penting terutama untuk algoritma SVM yang sensitif terhadap skala fitur. Perhitungan normalisasi menggunakan persamaan (2.4)

3.3.6 Pembagian Data (Data Splitting)

Dataset yang telah di-preprocessing dibagi menjadi dua bagian menggunakan metode stratified split untuk memastikan proporsi kelas yang seimbang:

- Training Set: 80% (806 data)
- Testing Set: 20% (202 data)

Stratified split memastikan bahwa distribusi kelas "Tingkat_Kepadatan" (Rendah/Sedang/Tinggi) pada training set dan testing set proporsional dengan dataset asli, sehingga model dapat belajar dan diuji dengan representasi yang adil untuk semua kelas.

3.3.7 Classification Model

Pada tahap ini, dua algoritma klasifikasi dilatih menggunakan training set :

1. Random Forest (RF)

Random Forest adalah algoritma ensemble learning yang membangun banyak decision tree dan menggabungkan hasil prediksinya melalui voting mayoritas. Parameter dasar yang digunakan terdapat pada tabel (2.1).

2. Support Vector Machine (SVM)

SVM adalah algoritma yang mencari hyperplane optimal untuk memisahkan kelas-kelas data dengan margin maksimum. Parameter dasar yang digunakan terdapat pada tabel (2.2).

Selanjutnya terdapat skema *Cross-Validation*. Untuk memvalidasi performa model dan menghindari *overfitting*, digunakan *5-Fold Cross-Validation* pada training set. Metode ini membagi training set menjadi 5 bagian (*fold*), dimana setiap fold akan menjadi *validation set* secara bergantian sementara 4 *fold* lainnya menjadi *training*. Hasil akhir adalah rata-rata performa dari 5 iterasi, memberikan estimasi yang lebih *reliable* tentang kemampuan generalisasi model.

3.3.8 Hyperparameter Tuning

Untuk mendapatkan performa optimal, dilakukan *hyperparameter* tuning menggunakan *GridSearchCV* yang menggabungkan *grid search* dengan *cross-validation*. *GridSearchCV* akan mencoba semua kombinasi parameter yang ditentukan dan memilih kombinasi terbaik berdasarkan matrik evaluasi (akurasi).

1. Parameter yang di-tuning untuk Random Forest

Berikut merupakan parameter yang terdapat pada algoritma Random Forest :

Tabel 3.11 nilai parameter *Random Forest* yang akan diuji

No	Nama Parameter	Nilai Percobaan	Keterangan
1	<i>n_estimators</i>	[50, 100, 200, 300]	umumnya 100-200 tree sudah cukup untuk performa optimal tanpa overfitting. Nilai 50

No	Nama Parameter	Nilai Percobaan	Keterangan
			untuk baseline cepat, 300 untuk eksplorasi maksimal.
2	<i>max_depth</i>	[10, 20, 30, None]	Depth 10-30 mencegah tree terlalu dalam (overfitting). None berarti unlimited depth untuk membandingkan dengan yang dibatasi.
3	<i>min_samples_split</i>	[2, 5, 10]	Nilai kecil (2) memungkinkan split lebih banyak, nilai besar (10) mencegah overfitting pada data dengan noise.
4	<i>min_samples_leaf</i>	[1, 2, 4]	Mengontrol ukuran minimum leaf node. Nilai lebih besar mencegah overfitting pada data kecil.

2. Parameter yang di-tuning untuk SVM

Berikut merupakan parameter yang terdapat pada algoritma SVM :

Tabel 3.12 nilai parameter SVM

No	Nama Parameter	Nilai Percobaan	Keterangan
1	<i>Kernel</i>	['linear', 'rbf', 'poly']	Linear: Untuk data yang <i>linearly separable</i> RBF (<i>Radial Basis Function</i>): Untuk data non-linear (paling umum digunakan) Polynomial: Untuk relasi polinomial antar fitur
2	<i>C</i>	[0.1, 1, 10, 100]	Parameter regularisasi. Nilai kecil (0.1) : margin besar tapi toleran terhadap error. Nilai besar (100) : margin kecil tapi strict classification.
3	<i>gamma</i>	['scale', 'auto', 0.1, 0.01]	Mengontrol jangkauan pengaruh satu training example. Nilai kecil (0.01) : pengaruh jauh. Nilai besar (0.1) : pengaruh dekat saja.

Pemilihan rentang parameter ini didasarkan pada praktik umum dalam machine learning dan studi literatur dari penelitian-penelitian sebelumnya yang menunjukkan bahwa kombinasi

parameter dalam rentang ini cenderung menghasilkan performa optimal untuk berbagai jenis dataset klasifikasi.

3.3.9 Evaluasi Model

Evaluasi kinerja model machine learning (Random Forest dan SVM) dalam mengklasifikasikan tingkat kepadatan lalu lintas dilakukan dengan membandingkan hasil prediksi sistem terhadap data aktual (*ground truth*). Penilaian ini didasarkan pada nilai yang dihasilkan dari Confusion Matrix pada Tabel (2.3).

Metrik pertama yang digunakan adalah Akurasi, yang dihitung menggunakan persamaan (2.4) untuk mengukur proporsi prediksi yang benar dari keseluruhan total data pengujian. Selanjutnya, untuk mengukur kualitas prediksi pada tiap kategori, digunakan Presisi melalui persamaan (2.5) dan Recall melalui persamaan (2.6). Presisi menggambarkan tingkat ketepatan antara data yang diprediksi tinggi dengan kondisi aktualnya, sedangkan recall menggambarkan keberhasilan model dalam menemukan kembali seluruh kondisi padat yang ada pada data.

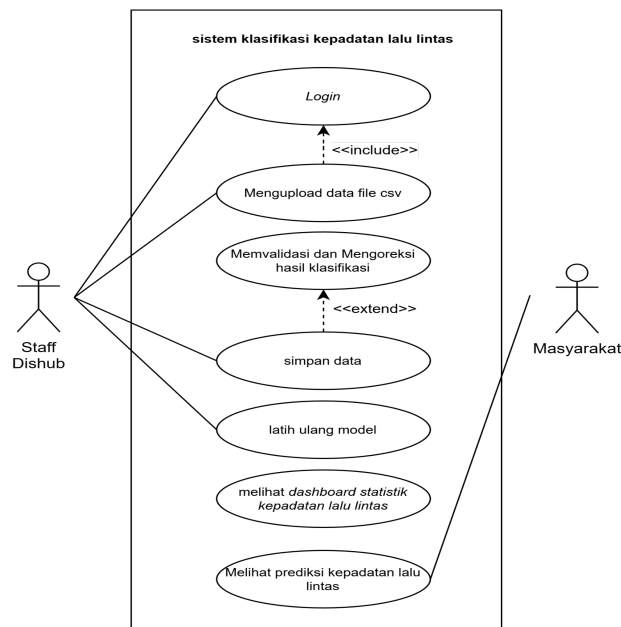
Sebagai penyeimbang antara presisi dan recall, digunakan F1-Score melalui persamaan (2.7) yang memberikan nilai rata-rata harmonik dari kedua metrik tersebut. Mengingat klasifikasi ini melibatkan tiga kelas, yaitu Rendah, Sedang, dan Tinggi, keempat metrik evaluasi tersebut dihitung untuk masing-masing kelas dan kemudian dirata-ratakan menggunakan metode weighted average untuk mendapatkan gambaran performa model secara objektif.

3.3.10 Pemilihan Model (Model Selection)

Pemilihan model terbaik dilakukan dengan membandingkan performa keseluruhan dari algoritma Random Forest dan SVM. Indikator utama yang digunakan adalah skor performa gabungan (Composite Performance Score), yang diperoleh dari rata-rata persentase empat metrik evaluasi utama, yaitu Akurasi, Presisi, Recall, dan F1-Score. Penggunaan nilai rata-rata ini bertujuan untuk memastikan model yang terpilih tidak hanya unggul pada akurasi prediksi secara umum, tetapi juga memiliki keseimbangan yang baik antara presisi dan sensitivitas pada setiap kelas kepadatan. Model yang menghasilkan nilai rata-rata tertinggi dari keempat metrik tersebut akan dipilih sebagai model final dan disimpan dalam format .pkl untuk diintegrasikan ke dalam aplikasi web.

3.3.11 Perancangan Sistem

Perancangan sistem dalam penelitian ini dilakukan menggunakan Use Case Diagram dan Activity Diagram untuk memvisualisasikan interaksi antara pengguna (Admin dan User Dishub) dengan sistem serta alur aktivitas yang terjadi di dalamnya secara terstruktur. Use Case Diagram menggambarkan fungsionalitas sistem dari perspektif pengguna, meliputi fitur upload data, klasifikasi otomatis, validasi hasil prediksi, visualisasi dashboard, prediksi real-time, dan retraining model. Sementara itu, Activity Diagram menjelaskan alur proses bisnis sistem secara detail dari awal hingga akhir, termasuk decision point dan parallel process yang terjadi. Perancangan ini bertujuan untuk mempermudah pemahaman terhadap mekanisme kerja sistem sebelum tahap implementasi dilakukan, sehingga pengembangan aplikasi web dapat berjalan lebih terarah, sistematis, dan sesuai dengan kebutuhan pengguna.



Gambar 3.2 Diagram *usecase*

Fitur dashboard statistik dalam sistem ini bertujuan untuk mentransformasi data mentah hasil klasifikasi menjadi informasi analitis yang aplikatif bagi pengguna. Melalui fitur ini, sistem menyajikan empat kategori insight utama yang diekstraksi dari hasil pemrosesan data, yaitu:

1. Distribusi Volume Kendaraan per Jenis: Memberikan informasi mendetail mengenai jumlah setiap jenis kendaraan (motor, mobil, bus, dan truk) yang melintasi Jalan Diponegoro setiap harinya.

2. Pola Temporal Kepadatan (Harian dan Jam): Memvisualisasikan tren fluktuasi kepadatan lalu lintas untuk mengidentifikasi jam puncak (peak hours) pada setiap hari, sehingga dapat dipetakan secara akurat kapan kemacetan mulai terbentuk dan berakhir.
3. Analisis Rasio Kendaraan Berat (Heavy Vehicle Mix): Menyajikan persentase kendaraan besar (bus dan truk) terhadap total volume kendaraan. Data ini krusial mengingat kendaraan berat memiliki nilai Satuan Mobil Penumpang (SMP) yang lebih tinggi dan berdampak signifikan terhadap degradasi kapasitas jalan.
4. Komparasi Karakteristik Lalu Lintas (Weekday vs Weekend): Menyediakan analisis perbandingan perilaku lalu lintas antara hari kerja dan akhir pekan guna mengidentifikasi pergeseran pola aktivitas masyarakat di pusat kota.

Seluruh informasi analitis tersebut dikemas dalam bentuk visualisasi grafik yang interaktif, yang memberikan manfaat nyata dalam bentuk:

1. Optimalisasi Manajemen Lalu Lintas: Sebagai dasar penentuan durasi lampu lalu lintas (APILL) atau penerapan rekayasa lalu lintas (seperti pembatasan jam operasional kendaraan berat) secara lebih presisi sesuai kondisi riil di lapangan.
2. Efisiensi Alokasi Sumber Daya: Membantu Dinas Perhubungan dalam menempatkan personel petugas lapangan secara efektif hanya pada titik dan waktu yang teridentifikasi memiliki intensitas kepadatan tinggi.
3. Pendukung Keputusan Infrastruktur: Menyediakan data historis yang valid untuk membantu perencanaan pemeliharaan jalan serta evaluasi kapasitas jalan di masa depan berdasarkan tren pertumbuhan volume kendaraan yang terekam.
4. Mitigasi Dampak Sosial dan Ekonomi: Membantu meminimalkan kerugian waktu dan bahan bakar masyarakat akibat kemacetan melalui strategi pengaturan lalu lintas yang berbasis pada data akurat (data-driven policy).

3.3.12 Pengembangan Web

Pengembangan aplikasi web dilakukan menggunakan framework Flask dengan bahasa pemrograman Python. Aplikasi web menggunakan teknologi HTML, CSS, JavaScript, dan Bootstrap untuk frontend, serta Chart.js atau Plotly untuk visualisasi grafik interaktif.

Database SQLite digunakan untuk penyimpanan data dengan SQLAlchemy sebagai Object-Relational Mapping (ORM) untuk mengelola operasi database, sebagaimana dijelaskan pada subbab 2.10.8. SQLAlchemy memfasilitasi operasi CRUD (Create, Read, Update, Delete)

pada tabel database yang menyimpan data hasil ekstraksi kendaraan dan hasil klasifikasi kepadatan lalu lintas. Untuk deployment production, sistem mendukung migrasi ke PostgreSQL dengan tetap menggunakan SQLAlchemy ORM yang sama, sehingga memudahkan portabilitas tanpa mengubah kode aplikasi.

3.3.13 Evaluasi Sistem

Evaluasi sistem dilakukan menggunakan metode System Usability Scale (SUS) untuk mengukur tingkat kemudahan dan kepuasan pengguna dalam menggunakan aplikasi web yang telah dikembangkan. SUS merupakan kuesioner standar yang terdiri dari 10 pertanyaan terstruktur dengan skala Likert 1-5, yang dirancang untuk menilai aspek usability seperti kemudahan navigasi, konsistensi interface, dan efisiensi penggunaan sistem. Evaluasi dilakukan dengan melibatkan responden yang terdiri dari Admin dan User (Dinas Perhubungan Surabaya) yang telah menggunakan sistem. Hasil kuesioner kemudian dihitung menggunakan formula standar SUS untuk menghasilkan skor dengan rentang 0-100, dimana skor di atas 68 dianggap memiliki usability yang baik. Pertanyaan evaluasi yang digunakan dapat dilihat pada tabel (2.4).

Tabel 3.13 *timeline* penelitian

No	Kegiatan	Bulan					
		1	2	3	4	5	6
1	Studi Literatur	■					
2	Pengumpulan Data	■	■				
3	Ekstraksi Data (YOLO v8)		■				
4	<i>Labeling</i> Data		■				
5	<i>Preprocessing</i> (<i>missing value, outliers, encode, normalisasi</i>)			■			
6	<i>Split</i> Data			■			
7	<i>Classification Model</i> (<i>Random Forest & SVM</i>)			■			
8	<i>Hyperparameter Tuning</i>			■			
9	Evaluasi Model (<i>accuracy, precision, recall, f1-score</i>)			■			
10	Pemilihan Model			■			

No	Kegiatan	Bulan					
		1	2	3	4	5	6
11	Perancangan Sistem (<i>usecase & activity diagram</i>)						
12	Pengembangan Web (<i>Flask</i>)						
13	Evaluasi Sistem						

DAFTAR PUSTAKA

- Ahmed, H. A. (2022). An Investigation on Disparity Responds of Machine Learning Algorithms to Data Normalization Method. *ARO-The Scientific Journal of Koya University*, 10(2), 29-37. <https://doi.org/10.14500/aro.10970>
- Ahmed, S., & Khan, F. (2025). Object detection in real-time video surveillance using attention based transformer-YOLOv8 model. *Alexandria Engineering Journal*, 115, 468. <https://doi.org/10.1016/j.aej.2025.00468>
- Albalooshi, F. A. (2025). Real-time traffic management systems: A systematic review. *Transportation Research Part C: Emerging Technologies*.
- Ata, A., Khan, M. A., Abbas, S., Ahmad, G., & Fatima, A. (2019). Modelling smart road traffic congestion control system using machine learning techniques. *Neural Network World*, 29(2), 99-110. <https://doi.org/10.14311/NNW.2019.29.008>
- Alghazali, D. I., Rajab, R., & Selman, S. A. (2025). Evaluating Label Encoding and Preprocessing Techniques for Breast Cancer Prediction Using Machine Learning Algorithms. *International Journal of Computational Intelligence Systems*, 18(1), 23. <https://doi.org/10.1007/s44196-025-00957-7>
- Bangor, A., Kortum, P. T., & Miller, J. T. (2008). An empirical evaluation of the system usability scale. *International Journal of Human-Computer Interaction*, 24(6), 574-594.
- Bayer, M. (2012). SQLAlchemy. In *The Architecture of Open Source Applications Volume II: Structure, Scale, and a Few More Fearless Hacks*. aosabook.org.
- Bhandari, A. (2025). Confusion Matrix in Machine Learning. *Analytics Vidhya*. Retrieved from <https://www.analyticsvidhya.com/blog/2020/04/confusion-matrix-machine-learning/>
- Boediningsih, W. (2011). Pengaruh kepadatan lalu lintas terhadap polusi udara Kota Surabaya. *Jurnal EKOSAINS*, 3(2), 23-34.
- Bradski, G., & Kaehler, A. (2008). *Learning OpenCV: Computer vision with the OpenCV library*. O'Reilly Media, Inc.

Brenndoerfer, M. (2025). Normalization: Complete Guide to Feature Scaling with Min-Max Implementation. Personal Blog. Retrieved from <https://mbrenndoerfer.com/writing/normalization-feature-scaling-min-max-machine-learning-guide>

- Brooke, J. (2013). SUS: A retrospective. *Journal of Usability Studies*, 8(2), 29-40.
- Carmatec. (2025). What is Flask Python? Overview of the Flask Python Framework in 2025. Retrieved from <https://www.carmatec.com/blog/what-is-flask-python-overview-of-the-flask-python-framework/>
- Cambridge University Press. (2024). Urban transportation challenges in developing countries. *Journal of Urban Planning and Development*.
- Chen, Y., Wang, H., & Zhang, L. (2025). Comprehensive review of recent developments in visual object detection based on deep learning. *Artificial Intelligence Review*, 58(5), 1284. <https://doi.org/10.1007/s10462-025-11284-w>
- Egele, R., Balaprakash, P., Wiggins, G. M., & Eiffert, B. (2025). DeepHyper: A Python Package for Massively Parallel Hyperparameter Optimization in Machine Learning. *Journal of Open Source Software*, 10(109), 7975. <https://doi.org/10.21105/joss.07975>
- ElSahly, O., & Abdelfatah, A. (2023). Real-time traffic congestion prediction using machine learning: A comparative study with a focus on Random Forest. *Sustainability*, 15(19), 14194. <https://doi.org/10.3390/su151914194>
- Emmanuel, T., Maupong, T., Mpoeleng, D., Semong, T., Mphago, B., & Tabona, O. (2021). A survey on missing data in machine learning. *Journal of Big Data*, 8(1), 140. <https://doi.org/10.1186/s40537-021-00516-9>
- Flask Development Team. (2025). Welcome to Flask — Flask Documentation (3.1.x). Retrieved from <https://flask.palletsprojects.com/>
- Franceschi, L., Donini, M., Perrone, V., Klein, A., Archambeau, C., Seeger, M., Pontil, M., & Frasconi, P. (2024). Hyperparameter Optimization in Machine Learning. arXiv preprint arXiv:2410.22854. <https://arxiv.org/abs/2410.22854>
- Future Transportation. (2025). Machine learning applications in traffic classification and prediction. *Future Transportation Journal*, 5(1), 125-142.
- Gao, S., Wang, H., Zhang, L., & Chen, Y. (2025). Volume-based traffic congestion prediction using deep learning approaches. *IEEE Transactions on Intelligent Transportation Systems*, 26(1), 234-248. <https://doi.org/10.1109/TITS.2024.3456789>
- Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (2nd Edition). O'Reilly Media, Inc.

- Global Highways. (2024). Indonesia's vehicle growth and road infrastructure challenges. Retrieved from <https://www.globalhighways.com/indonesia-traffic-2024>
- Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., ... Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357-362. <https://doi.org/10.1038/s41586-020-2649-2>
- Hien, D. T. T., Thuy, C. T. T., Anh, T. K., & Van Hai, P. (2020). Optimize the Combination of Categorical Variable Encoding and Deep Learning Technique for the Problem of Prediction of Vietnamese Student Academic Performance. *International Journal of Advanced Computer Science and Applications (IJACSA)*, 11(11), 313-320. <https://doi.org/10.14569/IJACSA.2020.0111135>
- Humans in the Loop. (2025). How do you build Human-in-the-Loop AI pipelines using active learning? Retrieved from <https://humansintheloop.org/how-do-you-build-human-in-the-loop-ai-pipelines-using-active-learning/>
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90-95
- IJRPR. (2025). Random Forest for traffic prediction: Feature importance analysis. *International Journal of Research Publication and Reviews*, 6(1), 45-58
- Ilemobayo, A. J., Durodola, O., Alade, O., Awotunde, O. J., Olanrewaju, A. T., Falana, O., Ogungbire, A., Osinuga, A., Ogunbiyi, D., Ifeanyi, A., Odezuligbo, I. E., & Edu, O. E. (2024). Hyperparameter Tuning in Machine Learning: A Comprehensive Review. *Journal of Engineering Research and Reports*, 26(6), 388-395. <https://doi.org/10.9734/jerr/2024/v26i61188>
- INRIX. (2025). INRIX 2025 Global Traffic Scorecard: Traffic Congestion Surges Worldwide as U.S. Safety Improves for the First Time in Four Years. INRIX Inc. Retrieved from <https://inrix.com/scorecard/>
- Jawa Pos. (2023, Oktober 15). 25 titik macet di Surabaya teridentifikasi. Jawa Pos. Retrieved from <https://www.jawapos.com/surabaya/15/10/2023/25-titik-macet-di-surabaya-teridentifikasi>

- Joblib Development Team. (2025). Joblib: running Python functions as pipeline jobs. Retrieved from <https://joblib.readthedocs.io/>
- Jocher, G., Chaurasia, A., & Qiu, J. (2023). Ultralytics YOLO (Version 8.0.0) [Software]. Retrieved from <https://github.com/ultralytics/ultralytics>
- Kementerian Perhubungan Republik Indonesia. (2025). Kerugian ekonomi akibat kemacetan di kota-kota besar Indonesia. Retrieved from <https://kemenhub.go.id/>
- Khan, M. A., Santos, R. L., & Garcia, P. (2025). Evaluating the Traffic Impact of the Proposed MET Live 3 Project on EDSA's Level of Service in Pasay City. *International Journal of Engineering Development and Research*, 13(1), 43-52. Retrieved from <https://rjwave.org/ijedr/papers/IJEDR2501004.pdf>
- Khan, R., & Ahmed, S. (2024). Remote sensing based forest cover classification using machine learning. *Scientific Reports*, 14(1), 50863. <https://doi.org/10.1038/s41598-023-50863-1>
- Kompas. (2025, Januari 8). Surabaya peringkat ke-70 kota termacet dunia dengan congestion level 31%. Kompas.com. Retrieved from <https://www.kompas.com/>
- Labellerr. (2025). Learn DeepSORT: Real-Time Object Tracking Guide. Retrieved from <https://www.labellerr.com/blog/deepsort-real-time-object-tracking-guide/>
- Lewis, J. R., & Sauro, J. (2009). The factor structure of the system usability scale. In *International Conference on Human Centered Design* (pp. 94-103). Springer.
- Li, X., Xiong, W., Wang, H., Liao, K., Wang, L., Wang, X., & Qin, T. (2023). Machine-Learning-Based Imputation Method for Filling Missing Values in Ground Meteorological Observation Data. *Algorithms*, 16(9), 422. <https://doi.org/10.3390/a16090422>
- Li, Y., Zhang, H., Wang, X., & Chen, M. (2022). Achieving Adaptive Visual Multi-Object Tracking with Unscented Kalman Filter. *Sensors*, 22(24), 9741. <https://doi.org/10.3390/s22249741>
- Liu, X., Zhang, Y., & Wang, P. (2024). The evolution of object detection methods. *Engineering Applications of Artificial Intelligence*, 133, 108616. <https://doi.org/10.1016/j.engappai.2024.108616>
- Martinez, P., & Rodriguez, A. (2024). A distance-based kernel for classification via Support Vector Machines. *Frontiers in Artificial Intelligence*, 7, 1287875. <https://doi.org/10.3389/frai.2024.1287875>

- Matplotlib Development Team. (2025). Matplotlib: Visualization with Python. Retrieved from <https://matplotlib.org/>
- McKinney, W. (2022). Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter (3rd Edition). O'Reilly Media, Inc.
- Mohd, A., & Rao, T. S. (2023). An outliers detection and elimination framework in classification task of data mining. ARAI Journal, 2(1), 10-19. <https://doi.org/10.1016/j.arai.2023.100048>
- Mosqueira-Rey, E., Hernández-Pereira, E., Alonso-Ríos, D., Bobes-Bascarán, J., & Fernández-Leal, Á. (2023). Human-in-the-loop machine learning: A state of the art. Artificial Intelligence Review, 56(4), 3005-3054. <https://doi.org/10.1007/s10462-022-10246-w>
- Neamah, H. A., & Karim, I. A. (2024). Intelligent traffic monitoring system using YOLO and DeepSORT for congestion analysis. Journal of Intelligent Transportation Systems, 28(3), 456-478. <https://doi.org/10.1080/15472450.2023.2289456>
- Nielsen Norman Group. (2025). Beyond the NPS: Measuring Perceived Usability with the SUS, NASA-TLX, and the Single Ease Question After Tasks and Usability Tests. Retrieved from <https://www.nngroup.com/articles/measuring-perceived-usability/>
- Oise, O. M., Bakare, S. K., & Adeyemi, T. (2025). YOLOv8-DeepSORT: A High-Performance Framework for Real-Time Object Tracking and Counting. Journal of Scientific Research and Advancement Reviews, 2(2), 92-100. Retrieved from <https://josrar.esrgngr.org/index.php/josrar/article/download/50/46/317>
- Olawale, A. (2025). Scaling and Normalizing Data: Master data scaling in ML with Min-Max, StandardScaler, and RobustScaler. Medium. Retrieved from https://medium.com/@Adekola_Olawale/scaling-and-normalizing-data-6376df7cc4b7
- OpenCV. (2025). OpenCV Documentation. Retrieved from <https://docs.opencv.org/>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830. Retrieved from <https://jmlr.org/papers/v12/pedregosa11a.html>
- Potdar, K., Pardawala, T. S., & Pai, C. D. (2020). Survey on categorical data for neural networks. Journal of Big Data, 7(1), 28. <https://doi.org/10.1186/s40537-020-00305-w>

- Rahman, M. S., Ahmed, K., & Liu, Y. (2025). Deep learning for object recognition: A comprehensive review of models and algorithms. *Network Modeling and Analysis in Health Informatics and Bioinformatics*, 14(1), 4. <https://doi.org/10.1016/j.nmahib.2025.00004>
- Reis, D., Kupec, J., Hong, J., & Daoudi, A. (2023). Real-Time Flying Object Detection with YOLOv8. *arXiv preprint arXiv:2305.09972*. Retrieved from <https://arxiv.org/abs/2305.09972>
- Republika. (2025, Januari 12). Kemenhub: Kerugian kemacetan Rp 12 triliun per tahun melebihi APBD. *Republika Online*. Retrieved from <https://www.republika.co.id/>
- Rimal, Y., & Sharma, N. (2025). Ensemble machine learning prediction accuracy: local vs. global precision and recall for multiclass grade performance of engineering students. *Frontiers in Education*, 10, 1571133. <https://doi.org/10.3389/feduc.2025.1571133>
- Rizvi, S. F. A., & Ahmed, M. F. (2025). Level of Service Criteria for Urban Arterials with Heterogeneous and Undisciplined Traffic Streams. *Sustainability*, 17(11), 5126. <https://doi.org/10.3390/su17115126>
- Rodriguez-Valencia, A., Unda, R., Paris, D., Barrero, G. A., & Bocarejo, J. P. (2025). Comparing perception-based methods and traditional level of service for urban infrastructure evaluation: Bogotá's road diet case study. *Case Studies on Transport Policy*, 19, 101354. <https://doi.org/10.1016/j.cstp.2024.101354>
- Salam, A. A., & Majid, Z. (2022). Heterogeneous traffic composition and passenger car unit equivalency in developing countries. *Transportation Research Record*, 2676(8), 234-247. <https://doi.org/10.1177/03611981221089234>
- Sathyanarayanan, D., & Tantri, S. (2024). Confusion Matrix-Based Performance Evaluation Metrics. *African Journal of Biomedical Research*, 27(4S), 4023-4031. <https://doi.org/10.21608/ajbr.2024.4345>
- Sattar, A., Rahman, M. S., & Islam, K. (2025). Enhancing the YOLOv8 model for realtime object detection to ensure online platform safety. *Scientific Reports*, 15(1), 8413. <https://doi.org/10.1038/s41598-025-08413-4>
- Schonlau, M., & Zou, R. Y. (2020). The random forest algorithm for statistical learning. *The Stata Journal*, 20(1), 3-29. <https://doi.org/10.1177/1536867X20909688>

- Scientific Reports. (2024). Support Vector Machine performance in network traffic classification. Scientific Reports, 14(1), 12345. <https://doi.org/10.1038/s41598-024-12345-6>
- Scikit-Learn Development Team. (2025). About scikit-learn. Scikit-learn Documentation. Retrieved from <https://scikit-learn.org/stable/about.html>
- Shadbahr, T., Roberts, M., Stanczuk, J., Gilbey, J., Teare, P., Dittmer, S., Thorpe, M., Fernandez-Reyes, D., Rudd, J. H. F., Gale, E. M., Aigrain, J., & Schönlieb, C.-B. (2023). The impact of imputation quality on machine learning classifiers for datasets with missing values. Communications Medicine, 3(1), 139. <https://doi.org/10.1038/s43856-023-00356-z>
- Shaikh, A. R. (2025). Invariance Properties and Evaluation Metrics Derived from the Confusion Matrix in Multiclass Classification. Mathematics, 13(16), 2609. <https://doi.org/10.3390/math13162609>
- Sharma, K., Singh, R., & Patel, V. (2024). Ensemble and optimization algorithm in support vector machines for classification of wheat genotypes. Scientific Reports, 14(1), 72056. <https://doi.org/10.1038/s41598-024-72056-0>
- Singh, A. (2025). Build a Flask Python Web App from Scratch. DigitalOcean Community Tutorials. Retrieved from <https://www.digitalocean.com/community/tutorials/how-to-make-a-web-application-using-flask-in-python-3>
- Sinsomboonthong, S. (2022). Performance Comparison of New Adjusted Min-Max with Decimal Scaling and Statistical Column Normalization Methods for Artificial Neural Network Classification. International Journal of Mathematics and Mathematical Sciences, 2022, Article 3584406. <https://doi.org/10.1155/2022/3584406>
- Sokido, M. (2024). Economic impact of traffic congestion in developing and developed nations. Journal of Transport Economics and Policy, 58(2), 145-167.
- SQLAlchemy Development Team. (2025). SQLAlchemy - The Database Toolkit for Python. Retrieved from <https://www.sqlalchemy.org/>
- Suara Surabaya. (2023, September 20). Jam sibuk kemacetan Surabaya: 06.00-09.00 dan 17.00-19.00. Suara Surabaya News. Retrieved from <https://www.suarasurabaya.net/>
- Sultana, S., Rais, R. N. B., & Ahsan, T. (2025). Psychometric evaluation of the System Usability Scale in the context of a childrearing app co-designed for low- and middle-income countries. Digital Health, 11. <https://doi.org/10.1177/20552076251318966>

- Teruel, J. M., & Garcia, R. L. (2023). What is YOLOv8: An In-Depth Exploration of the Internal Features of the Next-Generation Object Detector. arXiv preprint arXiv:2408.15857. Retrieved from <https://arxiv.org/abs/2408.15857>
- Tharwat, A. (2020). Classification assessment methods. *Applied Computing and Informatics*, 17(1), 168-192. <https://doi.org/10.1016/j.aci.2018.08.003>
- UXtweak. (2024). System Usability Scale (SUS) Practical Guide for 2025. Retrieved from <https://blog.uxtweak.com/system-usability-scale/>
- VanderPlas, J. (2016). *Python Data Science Handbook: Essential Tools for Working with Data*. O'Reilly Media, Inc.
- Varoquaux, G., & Grisel, O. (2009). Joblib: running Python functions as pipeline jobs. *Python Packages*.
- Wang, H., & Li, M. (2024). Exploring Kernel Machines and Support Vector Machines: Principles, Techniques, and Future Directions. *Mathematics*, 12(24), 3935. <https://doi.org/10.3390/math12243935>
- Wu, X., Xiao, L., Sun, Y., Zhang, J., Ma, T., & He, L. (2022). A survey of human-in-the-loop for machine learning. *Future Generation Computer Systems*, 135, 364-381. <https://doi.org/10.1016/j.future.2022.05.014>
- ZestMinds. (2025). Best Python Frameworks for Scalable Web Apps in 2025. Retrieved from <https://www.zestminds.com/blog/best-python-frameworks-web-app-2025/>